

# MINA

## Developer guide and source map

### Mediation, Integration and Automation

Version 1.0.3 - 96 activities, 11 groups, 85 functions and 12 shared connection types.

Repository navigation for every palette activity and core development feature. Source locations include verified search anchors and build-specific line numbers. Frontend, Python engine, native bridge and direct-export code paths are distinguished.

Use the linked contents or PDF bookmarks. Paths are relative to the repository root, not the installation directory. Edit maintained sources, add regression tests, rebuild and redistribute; do not patch generated archives as the permanent fix.

# Contents

|                                       |    |
|---------------------------------------|----|
| Developer guide: where to change MINA | 7  |
| Start: source map                     | 8  |
| Scheduler: source map                 | 11 |
| Call Sub Task: source map             | 13 |
| End: source map                       | 16 |
| Read File: source map                 | 19 |
| Write File: source map                | 22 |
| List Files: source map                | 24 |
| Delete File: source map               | 26 |
| Rename File: source map               | 28 |
| Copy File: source map                 | 30 |
| File Poller: source map               | 32 |
| FTP Get: source map                   | 34 |
| FTP Put: source map                   | 36 |
| FTP Delete: source map                | 38 |
| FTP Directory: source map             | 40 |
| FTP Change Directory: source map      | 42 |
| SFTP Get: source map                  | 44 |
| SFTP Put: source map                  | 46 |
| SFTP Delete: source map               | 48 |
| SFTP Directory: source map            | 50 |
| SFTP Change Directory: source map     | 52 |
| HTTP Listener: source map             | 54 |
| HTTP Request: source map              | 57 |
| HTTP Response: source map             | 59 |
| REST API Receiver: source map         | 62 |

|                                   |     |
|-----------------------------------|-----|
| REST API Invoke: source map       | 65  |
| SOAP Service: source map          | 68  |
| SOAP Request Reply: source map    | 71  |
| EMS Queue Receiver: source map    | 74  |
| EMS Topic Subscriber: source map  | 77  |
| EMS Queue Sender: source map      | 80  |
| EMS Topic Publisher: source map   | 83  |
| EMS Request Reply: source map     | 86  |
| EMS Reply: source map             | 89  |
| Get JMS Queue Message: source map | 92  |
| JMS Receive Message: source map   | 95  |
| JMS Request Reply: source map     | 98  |
| JMS Send Message: source map      | 101 |
| Reply to JMS Message: source map  | 104 |
| Wait for JMS Request: source map  | 107 |
| Kafka Receive Message: source map | 110 |
| Kafka Send Message: source map    | 112 |
| Kafka Get Messages: source map    | 114 |
| Pub/Sub Subscriber: source map    | 116 |
| Pub/Sub Publisher: source map     | 119 |
| Dynamic Connection: source map    | 122 |
| IDoc Acknowledgment: source map   | 125 |
| IDoc Confirmation: source map     | 128 |
| IDoc Converter: source map        | 131 |
| IDoc Listener: source map         | 134 |
| IDoc Parser: source map           | 137 |
| IDoc Reader: source map           | 140 |

|                                        |     |
|----------------------------------------|-----|
| Post IDoc to SAP: source map           | 143 |
| IDoc Renderer: source map              | 146 |
| RFC BAPI Listener: source map          | 149 |
| Invoke RFC BAPI in SAP: source map     | 152 |
| Reply from RFC BAPI in SAP: source map | 155 |
| Read Table: source map                 | 158 |
| JDBC Create: source map                | 161 |
| JDBC Update: source map                | 163 |
| JDBC Query: source map                 | 165 |
| JDBC Truncate: source map              | 167 |
| JDBC Delete: source map                | 169 |
| Stored Procedure: source map           | 171 |
| Dynamic SQL: source map                | 173 |
| Snowflake Insert: source map           | 175 |
| Snowflake Query: source map            | 177 |
| Snowflake Update: source map           | 179 |
| Snowflake Delete: source map           | 181 |
| Snowflake Bulk Load: source map        | 183 |
| AMQP Receive Message: source map       | 185 |
| AMQP Get Message: source map           | 188 |
| AMQP Send Message: source map          | 191 |
| AMQP Dead Letter Message: source map   | 193 |
| Parse XML: source map                  | 196 |
| Render XML: source map                 | 198 |
| Parse JSON: source map                 | 200 |
| Render JSON: source map                | 202 |
| Parse Data: source map                 | 204 |

|                                                    |     |
|----------------------------------------------------|-----|
| Render Data: source map                            | 206 |
| Read Excel Workbook: source map                    | 208 |
| Mapper: source map                                 | 210 |
| Transform: source map                              | 212 |
| Log: source map                                    | 214 |
| Catch Exception: source map                        | 216 |
| Throw Exception: source map                        | 218 |
| Rethrow Exception: source map                      | 220 |
| Confirm Message: source map                        | 222 |
| Java Activity: source map                          | 224 |
| Python Invoke: source map                          | 227 |
| External Command: source map                       | 229 |
| Assign: source map                                 | 231 |
| Checkpoint: source map                             | 233 |
| Sleep: source map                                  | 235 |
| Get Shared Variable: source map                    | 237 |
| Set Shared Variable: source map                    | 239 |
| Tasks, sub-tasks, persistence and copy             | 241 |
| Run, continuous events and Stop                    | 242 |
| Debug controls, test bench and Job Data            | 243 |
| Input mapping, schema mapper and suggestions       | 245 |
| Built-in and project functions                     | 247 |
| Canvas and mapper drag-and-drop                    | 248 |
| Groups: editor, graph boundaries and runtime state | 249 |
| If: source map                                     | 251 |
| While True: source map                             | 252 |
| For Each: source map                               | 253 |

|                                                 |     |
|-------------------------------------------------|-----|
| Iterate: source map                             | 254 |
| Repeat Until True: source map                   | 255 |
| Repeat on Error Until True: source map          | 256 |
| Scope: source map                               | 257 |
| None: source map                                | 258 |
| Critical Section: source map                    | 259 |
| JDBC Transaction: source map                    | 260 |
| Pick First: source map                          | 261 |
| Engine archives versus direct Python generation | 262 |
| Logs, faults and observable execution data      | 263 |
| Every built-in function: implementation index   | 264 |

# Developer guide: where to change MINA

Developer guide / Start here

## Architecture

This is a source-maintenance guide, separate from the activity user manual. Frontend code is React/TypeScript, core engine code is Python, and native connector work can execute in Java. Paths are relative to your repository checkout, not the installed program directory. Line numbers are regenerated from verified anchors and refer to this documentation build.

```
Palette / canvas -> ActivityEditor -> saved Project
Run API -> WorkflowRuntime.run -> execute_with_policy
-> resolve_activity_config -> execute -> adapter / native bridge
-> record_activity_output -> Run result / Debug Job Data

Raw Python export -> raw_python_files -> generated task modules
-> raw_python_support (specialized helpers) -> adapters / bridge
```

The source tables tell you where to start, not that every option is fully implemented. Read the operational guide for known behavior gaps. A passing unit test or this code index does not establish provider certification.

## How to navigate

Expand Developer guide > Activities for one page per palette activity. Features covers tasks, groups, Run, Debug, mapping, functions and drag-and-drop. Each source table provides a file:line plus a stable search anchor. Use your editor Go to File and Go to Line; if your checkout differs, search the anchor. The site deliberately does not expose source files or local filesystem URLs.

```
Example from repository root:
rg -n "def resolve_activity_config" backend/app/runtime.py
rg -n "file_activity" backend/app/raw_python_support/activities.py
```

Do not edit frontend/dist, installed app.asar, generated task archives or packaged engine copies as the permanent fix. Edit repository sources, regenerate/rebuild, and redistribute. Preserve saved activity IDs and config keys when changing labels.

## Change checklist

1. Decide whether the change is visual, schema, runtime, transport or export behavior. 2. Follow the relevant source map. 3. Preserve existing project defaults/aliases or add an explicit migration. 4. Update runtime and direct-export implementations where they differ. 5. Add normal, error, cancellation and legacy-project tests. 6. Regenerate docs, build and test a newly packaged install.

```
cd backend
python -m unittest discover -s tests
cd ../frontend
npm run docs:build
node --test scripts/documentation.test.mjs
npm run build
```

Use a development environment with the project dependencies installed. Provider tests require isolated test infrastructure and licensed drivers. Never use production queues, write APIs or secrets in documentation fixtures.

# Start: source map

Developer guide / Activities / Starters & Tasks

## Trace

Saved identity: type=start; config.operation=start. Start is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (start)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: start; select operation start                            | frontend/src/ActivityEditor.tsx:201<br>if (n.type === "start")   |
| Engine start dispatch; operation start                                    | backend/app/runtime.py:696<br>if activity.type == 'start':       |
| Direct helper dispatch for start; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Task graph and interface schema                                           | backend/app/models.py:307<br>class ProcessDefinition(            |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                           | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Start, select type=start and operation=start, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of start. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Scheduler: source map

Developer guide / Activities / Starters & Tasks

## Trace

Saved identity: type=timer; config.operation=schedule. Scheduler is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (timer)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: timer; select operation schedule                         | frontend/src/ActivityEditor.tsx:215<br>if (n.type === "timer")   |
| Engine timer dispatch; operation schedule                                 | backend/app/runtime.py:702<br>if activity.type == 'timer':       |
| Direct helper dispatch for timer; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                           | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Scheduler, select type=timer and operation=schedule, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of schedule. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Call Sub Task: source map

Developer guide / Activities / Starters & Tasks

## Trace

Saved identity: type=call\_task; config.operation=call. Call Sub Task is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (call_task)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                | Source location / search anchor                                    |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                             | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                                           | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                               | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: call_task; select operation call                             | frontend/src/ActivityEditor.tsx:243<br>if (n.type === "call_task") |
| Engine call_task dispatch; operation call                                     | backend/app/runtime.py:1116<br>if activity.type == 'call_task':    |
| Direct helper dispatch for call_task; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute(   |
| Task graph and interface schema                                               | backend/app/models.py:307<br>class ProcessDefinition(              |
| Direct graph/code generation                                                  | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                                   | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: tasks_runtime                                               | backend/tests/test_tasks_runtime.py:1<br>(module)                  |
| Regression suite: raw_python                                                  | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                               | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing Call Sub Task, select type=call\_task and operation=call, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of call. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# End: source map

Developer guide / Activities / Starters & Tasks

## Trace

Saved identity: type=end; config.operation=end. End is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (end)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                          | Source location / search anchor                                  |
|-------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                       | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                     | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                         | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: end; select operation end                              | frontend/src/ActivityEditor.tsx:208<br>if (n.type === "end")     |
| Engine end dispatch; operation end                                      | backend/app/runtime.py:699<br>if activity.type == 'end':         |
| Direct helper dispatch for end; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Task graph and interface schema                                         | backend/app/models.py:307<br>class ProcessDefinition(            |
| Direct graph/code generation                                            | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                             | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                         | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                            | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                         | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing End, select type=end and operation=end, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of end. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Read File: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=read. Read File is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation read                                                     | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation read                                             | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Text read implementation                                                                         | backend/app/runtime.py:1045<br>content = path.read_text(              |
| Binary read implementation                                                                       | backend/app/runtime.py:1044<br>path.read_bytes()                      |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

Example: adding a streaming/size-limit option requires a palette default if needed, a Configuration field in activityContract, mapping exposure via runtimeMappableInputs, a check before read\_text/read\_bytes in WorkflowRuntime.execute, and equivalent behavior in file\_activity for direct exports. Keep existing textContent/content/binaryContent/fileInfo keys compatible. Current reads materialize whole contents; changing the output to a stream changes the contract and cannot be a silent replacement.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of read. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Write File: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=write. Write File is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation write                                                    | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation write                                            | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing Write File, select type=file and operation=write, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of write. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# List Files: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=list. List Files is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation list                                                     | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation list                                             | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing List Files, select type=file and operation=list, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of list. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Delete File: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=delete. Delete File is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation delete                                                   | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation delete                                           | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing Delete File, select type=file and operation=delete, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of delete. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Rename File: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=rename. Rename File is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation rename                                                   | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation rename                                           | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing Rename File, select type=file and operation=rename, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of rename. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Copy File: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=copy. Copy File is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation copy                                                     | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation copy                                             | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing Copy File, select type=file and operation=copy, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of copy. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# File Poller: source map

Developer guide / Activities / File

## Trace

Saved identity: type=file; config.operation=poll. File Poller is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'file':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                                   | Source location / search anchor                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                                                                | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                                                              | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                                                                  | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: file; select operation poll                                                     | frontend/src/ActivityEditor.tsx:301<br>if (n.type === "file")         |
| Local file execution; read is the fall-through after write/list/delete/rename/copy/poll branches | backend/app/runtime.py:992<br>if activity.type == 'file':             |
| Direct Python implementation; inspect operation poll                                             | backend/app/raw_python_support/activities.py:50<br>def file_activity( |
| Direct graph/code generation                                                                     | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                                                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: activity_packs                                                                 | backend/tests/test_activity_packs.py:1<br>(module)                    |
| Regression suite: raw_python                                                                     | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                                                                  | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing File Poller, select type=file and operation=poll, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of poll. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# FTP Get: source map

Developer guide / Activities / FTP

## Trace

Saved identity: type=ftp; config.operation=get. FTP Get is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def ftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                   |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                    |
| FTP operations using the selected operation         | backend/app/runtime.py:1740<br>def ftp(                           |
| Direct Python implementation; inspect operation get | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                    | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing FTP Get, select type=ftp and operation=get, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of get. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# FTP Put: source map

Developer guide / Activities / FTP

## Trace

Saved identity: type=ftp; config.operation=put. FTP Put is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def ftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                   |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                    |
| FTP operations using the selected operation         | backend/app/runtime.py:1740<br>def ftp(                           |
| Direct Python implementation; inspect operation put | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                    | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing FTP Put, select type=ftp and operation=put, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of put. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# FTP Delete: source map

Developer guide / Activities / FTP

## Trace

Saved identity: type=ftp; config.operation=delete. FTP Delete is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def ftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                   |
|--------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                    |
| FTP operations using the selected operation            | backend/app/runtime.py:1740<br>def ftp(                           |
| Direct Python implementation; inspect operation delete | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                       | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing FTP Delete, select type=ftp and operation=delete, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of delete. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# FTP Directory: source map

Developer guide / Activities / FTP

## Trace

Saved identity: type=ftp; config.operation=dir. FTP Directory is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def ftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                   |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                    |
| FTP operations using the selected operation         | backend/app/runtime.py:1740<br>def ftp(                           |
| Direct Python implementation; inspect operation dir | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                    | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing FTP Directory, select type=ftp and operation=dir, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of dir. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# FTP Change Directory: source map

Developer guide / Activities / FTP

## Trace

Saved identity: type=ftp; config.operation=change\_dir. FTP Change Directory is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def ftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                             | Source location / search anchor                                   |
|------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                          | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                        | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                            | backend/app/debugger.py:274<br>async def step(                    |
| FTP operations using the selected operation                | backend/app/runtime.py:1740<br>def ftp(                           |
| Direct Python implementation; inspect operation change_dir | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                               | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                           | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                               | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                            | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing FTP Change Directory, select type=ftp and operation=change\_dir, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of change\_dir. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# SFTP Get: source map

Developer guide / Activities / SFTP

## Trace

Saved identity: type=sftp; config.operation=get. SFTP Get is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def sftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                   |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                    |
| SFTP operations using the selected operation        | backend/app/runtime.py:1759<br>def sftp(                          |
| Direct Python implementation; inspect operation get | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                    | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing SFTP Get, select type=sftp and operation=get, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of get. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# SFTP Put: source map

Developer guide / Activities / SFTP

## Trace

Saved identity: type=sftp; config.operation=put. SFTP Put is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def sftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                   |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                    |
| SFTP operations using the selected operation        | backend/app/runtime.py:1759<br>def sftp(                          |
| Direct Python implementation; inspect operation put | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                    | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing SFTP Put, select type=sftp and operation=put, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of put. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# SFTP Delete: source map

Developer guide / Activities / SFTP

## Trace

Saved identity: type=sftp; config.operation=delete. SFTP Delete is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def sftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                   |
|--------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                    |
| SFTP operations using the selected operation           | backend/app/runtime.py:1759<br>def sftp(                          |
| Direct Python implementation; inspect operation delete | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                       | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing SFTP Delete, select type=sftp and operation=delete, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of delete. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# SFTP Directory: source map

Developer guide / Activities / SFTP

## Trace

Saved identity: type=sftp; config.operation=dir. SFTP Directory is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def sftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                   |
|-----------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                    |
| SFTP operations using the selected operation        | backend/app/runtime.py:1759<br>def sftp(                          |
| Direct Python implementation; inspect operation dir | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                    | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing SFTP Directory, select type=sftp and operation=dir, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of dir. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# SFTP Change Directory: source map

Developer guide / Activities / SFTP

## Trace

Saved identity: type=sftp; config.operation=change\_dir. SFTP Change Directory is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def sftp(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                             | Source location / search anchor                                   |
|------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                          | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                        | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                            | backend/app/debugger.py:274<br>async def step(                    |
| SFTP operations using the selected operation               | backend/app/runtime.py:1759<br>def sftp(                          |
| Direct Python implementation; inspect operation change_dir | backend/app/raw_python_support/activities.py:210<br>def transfer( |
| Direct graph/code generation                               | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: activity_packs                           | backend/tests/test_activity_packs.py:1<br>(module)                |
| Regression suite: raw_python                               | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                            | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing SFTP Change Directory, select type=sftp and operation=change\_dir, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of change\_dir. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# HTTP Listener: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=http\_listener; config.operation=listen. HTTP Listener is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (http_listener)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                    | Source location / search anchor                                         |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Activity dispatch                                                                 | backend/app/runtime.py:694<br>async def execute(self,                   |
| Publish outputs and captured inputs                                               | backend/app/runtime.py:550<br>def record_activity_output(               |
| Debug stepping over the runtime                                                   | backend/app/debugger.py:274<br>async def step(                          |
| Contract branch: http_listener; select operation listen                           | frontend/src/ActivityEditor.tsx:412<br>if (n.type === "http_listener")  |
| Engine http_listener dispatch; operation listen                                   | backend/app/runtime.py:694<br>async def execute(self,                   |
| Direct helper dispatch for http_listener; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute(        |
| Inbound HTTP/REST/SOAP hosting and response boundary                              | backend/app/main.py:1860<br>async def invoke_listener(                  |
| Direct-export application hosting                                                 | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Direct graph/code generation                                                      | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Export capability selection                                                       | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Regression suite: tasks_runtime                                                   | backend/tests/test_tasks_runtime.py:1<br>(module)                       |
| Regression suite: raw_python                                                      | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export                                                   | backend/tests/test_engine_export.py:1<br>(module)                       |

## What to change

When changing HTTP Listener, select type=http\_listener and operation=listen, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of listen. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# HTTP Request: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=http; config.operation=request. HTTP Request is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'http':
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                          | Source location / search anchor                                       |
|---------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                       | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                     | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                         | backend/app/debugger.py:274<br>async def step(                        |
| HTTP client request                                     | backend/app/runtime.py:970<br>if activity.type == 'http':             |
| Direct Python implementation; inspect operation request | backend/app/raw_python_support/activities.py:250<br>def http_request( |
| Direct graph/code generation                            | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                             | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: tasks_runtime                         | backend/tests/test_tasks_runtime.py:1<br>(module)                     |
| Regression suite: raw_python                            | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                         | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing HTTP Request, select type=http and operation=request, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of request. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# HTTP Response: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=http\_response; config.operation=response. HTTP Response is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (http_response)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                                    | Source location / search anchor                                         |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Activity dispatch                                                                 | backend/app/runtime.py:694<br>async def execute(self,                   |
| Publish outputs and captured inputs                                               | backend/app/runtime.py:550<br>def record_activity_output(               |
| Debug stepping over the runtime                                                   | backend/app/debugger.py:274<br>async def step(                          |
| Contract branch: http_response; select operation response                         | frontend/src/ActivityEditor.tsx:491<br>if (n.type === "http_response")  |
| Engine http_response dispatch; operation response                                 | backend/app/runtime.py:813<br>if activity.type == 'http_response':      |
| Direct helper dispatch for http_response; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute(        |
| Inbound HTTP/REST/SOAP hosting and response boundary                              | backend/app/main.py:1860<br>async def invoke_listener(                  |
| Direct-export application hosting                                                 | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Direct graph/code generation                                                      | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Export capability selection                                                       | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Regression suite: tasks_runtime                                                   | backend/tests/test_tasks_runtime.py:1<br>(module)                       |
| Regression suite: raw_python                                                      | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export                                                   | backend/tests/test_engine_export.py:1<br>(module)                       |

## What to change

When changing HTTP Response, select type=http\_response and operation=response, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of response. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# REST API Receiver: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=rest; config.operation=receiver. REST API Receiver is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'rest' and
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                             | Source location / search anchor                                         |
|------------------------------------------------------------|-------------------------------------------------------------------------|
| Activity dispatch                                          | backend/app/runtime.py:694<br>async def execute(self,                   |
| Publish outputs and captured inputs                        | backend/app/runtime.py:550<br>def record_activity_output(               |
| Debug stepping over the runtime                            | backend/app/debugger.py:274<br>async def step(                          |
| REST outbound request; inbound path enters invoke_listener | backend/app/runtime.py:977<br>if activity.type == 'rest' and            |
| Direct Python implementation; inspect operation receiver   | backend/app/raw_python_support/activities.py:250<br>def http_request(   |
| Inbound HTTP/REST/SOAP hosting and response boundary       | backend/app/main.py:1860<br>async def invoke_listener(                  |
| Direct-export application hosting                          | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Direct graph/code generation                               | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Export capability selection                                | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Regression suite: tasks_runtime                            | backend/tests/test_tasks_runtime.py:1<br>(module)                       |
| Regression suite: raw_python                               | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export                            | backend/tests/test_engine_export.py:1<br>(module)                       |

## What to change

When changing REST API Receiver, select type=rest and operation=receiver, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of receiver. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# REST API Invoke: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=rest; config.operation=invoke. REST API Invoke is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'rest' and
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                             | Source location / search anchor                                         |
|------------------------------------------------------------|-------------------------------------------------------------------------|
| Activity dispatch                                          | backend/app/runtime.py:694<br>async def execute(self,                   |
| Publish outputs and captured inputs                        | backend/app/runtime.py:550<br>def record_activity_output(               |
| Debug stepping over the runtime                            | backend/app/debugger.py:274<br>async def step(                          |
| REST outbound request; inbound path enters invoke_listener | backend/app/runtime.py:977<br>if activity.type == 'rest' and            |
| Direct Python implementation; inspect operation invoke     | backend/app/raw_python_support/activities.py:250<br>def http_request(   |
| Inbound HTTP/REST/SOAP hosting and response boundary       | backend/app/main.py:1860<br>async def invoke_listener(                  |
| Direct-export application hosting                          | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Direct graph/code generation                               | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Export capability selection                                | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Regression suite: tasks_runtime                            | backend/tests/test_tasks_runtime.py:1<br>(module)                       |
| Regression suite: raw_python                               | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export                            | backend/tests/test_engine_export.py:1<br>(module)                       |

## What to change

When changing REST API Invoke, select type=rest and operation=invoke, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of invoke. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# SOAP Service: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=soap; config.operation=service. SOAP Service is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'soap' and
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                             | Source location / search anchor                                         |
|------------------------------------------------------------|-------------------------------------------------------------------------|
| Activity dispatch                                          | backend/app/runtime.py:694<br>async def execute(self,                   |
| Publish outputs and captured inputs                        | backend/app/runtime.py:550<br>def record_activity_output(               |
| Debug stepping over the runtime                            | backend/app/debugger.py:274<br>async def step(                          |
| Contract branch: soap; select operation service            | frontend/src/ActivityEditor.tsx:543<br>if (n.type === "soap")           |
| SOAP outbound request; service path enters invoke_listener | backend/app/runtime.py:984<br>if activity.type == 'soap' and            |
| Direct Python implementation; inspect operation service    | backend/app/raw_python_support/activities.py:250<br>def http_request(   |
| Inbound HTTP/REST/SOAP hosting and response boundary       | backend/app/main.py:1860<br>async def invoke_listener(                  |
| Direct-export application hosting                          | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Direct graph/code generation                               | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Export capability selection                                | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Regression suite: tasks_runtime                            | backend/tests/test_tasks_runtime.py:1<br>(module)                       |
| Regression suite: raw_python                               | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export                            | backend/tests/test_engine_export.py:1<br>(module)                       |

## What to change

When changing SOAP Service, select type=soap and operation=service, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of service. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# SOAP Request Reply: source map

Developer guide / Activities / HTTP & APIs

## Trace

Saved identity: type=soap; config.operation=request\_reply. SOAP Request Reply is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: if activity.type == 'soap' and
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                   | Source location / search anchor                                         |
|------------------------------------------------------------------|-------------------------------------------------------------------------|
| Activity dispatch                                                | backend/app/runtime.py:694<br>async def execute(self,                   |
| Publish outputs and captured inputs                              | backend/app/runtime.py:550<br>def record_activity_output(               |
| Debug stepping over the runtime                                  | backend/app/debugger.py:274<br>async def step(                          |
| Contract branch: soap; select operation<br>request_reply         | frontend/src/ActivityEditor.tsx:543<br>if (n.type === "soap")           |
| SOAP outbound request; service path<br>enters invoke_listener    | backend/app/runtime.py:984<br>if activity.type == 'soap' and            |
| Direct Python implementation; inspect<br>operation request_reply | backend/app/raw_python_support/activities.py:250<br>def http_request(   |
| Inbound HTTP/REST/SOAP hosting and<br>response boundary          | backend/app/main.py:1860<br>async def invoke_listener(                  |
| Direct-export application hosting                                | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Direct graph/code generation                                     | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Export capability selection                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Regression suite: tasks_runtime                                  | backend/tests/test_tasks_runtime.py:1<br>(module)                       |
| Regression suite: raw_python                                     | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export                                  | backend/tests/test_engine_export.py:1<br>(module)                       |

## What to change

When changing SOAP Request Reply, select type=soap and operation=request\_reply, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of request\_reply. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# EMS Queue Receiver: source map

Developer guide / Activities / EMS

## Trace

Saved identity: type=ems; config.operation=queue\_receiver. EMS Queue Receiver is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                 | Source location / search anchor                                   |
|----------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                              | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                            | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                                | backend/app/debugger.py:274<br>async def step(                    |
| EMS operation routing and body/ack handling                    | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation queue_receiver | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                 | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                           | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                   | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                    | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                      | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                   | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                                | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing EMS Queue Receiver, select type=ems and operation=queue\_receiver, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of queue\_receiver. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# EMS Topic Subscriber: source map

Developer guide / Activities / EMS

## Trace

Saved identity: type=ems; config.operation=topic\_subscriber. EMS Topic Subscriber is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                   | Source location / search anchor                                   |
|------------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                                | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                              | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                                  | backend/app/debugger.py:274<br>async def step(                    |
| EMS operation routing and body/ack handling                      | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation topic_subscriber | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                   | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                             | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                     | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                      | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                        | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                     | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                                  | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing EMS Topic Subscriber, select type=ems and operation=topic\_subscriber, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of topic\_subscriber. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# EMS Queue Sender: source map

Developer guide / Activities / EMS

## Trace

Saved identity: type=ems; config.operation=send. EMS Queue Sender is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                       | Source location / search anchor                                   |
|------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                    | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                  | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                      | backend/app/debugger.py:274<br>async def step(                    |
| EMS operation routing and body/ack handling          | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation send | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                       | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                 | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                         | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                          | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance            | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                         | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                      | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing EMS Queue Sender, select type=ems and operation=send, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of send. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# EMS Topic Publisher: source map

Developer guide / Activities / EMS

## Trace

Saved identity: type=ems; config.operation=publish. EMS Topic Publisher is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                          | Source location / search anchor                                   |
|---------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                       | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                     | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                         | backend/app/debugger.py:274<br>async def step(                    |
| EMS operation routing and body/ack handling             | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation publish | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                          | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                    | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                            | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                             | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance               | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                            | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                         | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing EMS Topic Publisher, select type=ems and operation=publish, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of publish. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# EMS Request Reply: source map

Developer guide / Activities / EMS

## Trace

Saved identity: type=ems; config.operation=request\_reply. EMS Request Reply is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                | Source location / search anchor                                   |
|---------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                             | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                           | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                               | backend/app/debugger.py:274<br>async def step(                    |
| EMS operation routing and body/ack handling                   | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation request_reply | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                          | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                  | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                   | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                     | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                  | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                               | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing EMS Request Reply, select type=ems and operation=request\_reply, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of request\_reply. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# EMS Reply: source map

Developer guide / Activities / EMS

## Trace

Saved identity: type=ems; config.operation=reply. EMS Reply is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                        | Source location / search anchor                                   |
|-------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                     | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                   | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                       | backend/app/debugger.py:274<br>async def step(                    |
| EMS operation routing and body/ack handling           | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation reply | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                        | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                  | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                          | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                           | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance             | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                          | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                       | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing EMS Reply, select type=ems and operation=reply, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of reply. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Get JMS Queue Message: source map

Developer guide / Activities / JMS

## Trace

Saved identity: type=jms; config.operation=get\_queue\_message. Get JMS Queue Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                    | Source location / search anchor                                   |
|-------------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                                 | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                               | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                                   | backend/app/debugger.py:274<br>async def step(                    |
| JMS operation routing and body/ack handling                       | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation get_queue_message | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                    | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                              | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                      | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                       | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                         | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                      | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                                   | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing Get JMS Queue Message, select type=jms and operation=get\_queue\_message, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of get\_queue\_message. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# JMS Receive Message: source map

Developer guide / Activities / JMS

## Trace

Saved identity: type=jms; config.operation=receive\_message. JMS Receive Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                  | Source location / search anchor                                   |
|-----------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                               | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                             | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                                 | backend/app/debugger.py:274<br>async def step(                    |
| JMS operation routing and body/ack handling                     | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation receive_message | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                  | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                            | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                    | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                     | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                       | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                    | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                                 | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing JMS Receive Message, select type=jms and operation=receive\_message, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of receive\_message. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# JMS Request Reply: source map

Developer guide / Activities / JMS

## Trace

Saved identity: type=jms; config.operation=request\_reply. JMS Request Reply is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                | Source location / search anchor                                   |
|---------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                             | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                           | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                               | backend/app/debugger.py:274<br>async def step(                    |
| JMS operation routing and body/ack handling                   | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation request_reply | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                          | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                  | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                   | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                     | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                  | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                               | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing JMS Request Reply, select type=jms and operation=request\_reply, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of request\_reply. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# JMS Send Message: source map

Developer guide / Activities / JMS

## Trace

Saved identity: type=jms; config.operation=send\_message. JMS Send Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                               | Source location / search anchor                                   |
|--------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                            | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                          | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                              | backend/app/debugger.py:274<br>async def step(                    |
| JMS operation routing and body/ack handling                  | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation send_message | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                               | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                         | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                 | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                  | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                    | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                 | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                              | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing JMS Send Message, select type=jms and operation=send\_message, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of send\_message. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Reply to JMS Message: source map

Developer guide / Activities / JMS

## Trace

Saved identity: type=jms; config.operation=reply\_message. Reply to JMS Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                | Source location / search anchor                                   |
|---------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                             | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                           | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                               | backend/app/debugger.py:274<br>async def step(                    |
| JMS operation routing and body/ack handling                   | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation reply_message | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                                | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                          | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                  | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                   | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                     | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                  | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                               | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing Reply to JMS Message, select type=jms and operation=reply\_message, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of reply\_message. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Wait for JMS Request: source map

Developer guide / Activities / JMS

## Trace

Saved identity: type=jms; config.operation=wait\_request. Wait for JMS Request is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                               | Source location / search anchor                                   |
|--------------------------------------------------------------|-------------------------------------------------------------------|
| Activity dispatch                                            | backend/app/runtime.py:694<br>async def execute(self,             |
| Publish outputs and captured inputs                          | backend/app/runtime.py:550<br>def record_activity_output(         |
| Debug stepping over the runtime                              | backend/app/debugger.py:274<br>async def step(                    |
| JMS operation routing and body/ack handling                  | backend/app/runtime.py:1143<br>async def messaging(               |
| Direct Python implementation; inspect operation wait_request | backend/app/raw_python_support/connectors.py:44<br>async def jms( |
| Python/native worker lifecycle                               | backend/app/java_bridge.py:1<br>(module)                          |
| Native Java connector implementation                         | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module) |
| Direct graph/code generation                                 | backend/app/raw_python.py:576<br>def raw_python_files(            |
| Export capability selection                                  | backend/app/raw_python.py:60<br>def _activity_capabilities(       |
| Regression suite: jms_publish_performance                    | backend/tests/test_jms_publish_performance.py:1<br>(module)       |
| Regression suite: raw_python                                 | backend/tests/test_raw_python.py:1<br>(module)                    |
| Regression suite: engine_export                              | backend/tests/test_engine_export.py:1<br>(module)                 |

## What to change

When changing Wait for JMS Request, select type=jms and operation=wait\_request, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of wait\_request. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_jms_publish_performance tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Kafka Receive Message: source map

Developer guide / Activities / Kafka

## Trace

Saved identity: type=kafka; config.operation=receive. Kafka Receive Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                          | Source location / search anchor                                      |
|---------------------------------------------------------|----------------------------------------------------------------------|
| Activity dispatch                                       | backend/app/runtime.py:694<br>async def execute(self,                |
| Publish outputs and captured inputs                     | backend/app/runtime.py:550<br>def record_activity_output(            |
| Debug stepping over the runtime                         | backend/app/debugger.py:274<br>async def step(                       |
| Contract branch: kafka; select operation receive        | frontend/src/ActivityEditor.tsx:783<br>if (n.type === "kafka")       |
| Kafka producer/consumer operation dispatch              | backend/app/runtime.py:1143<br>async def messaging(                  |
| Direct Python implementation; inspect operation receive | backend/app/raw_python_support/connectors.py:228<br>async def kafka( |
| Direct graph/code generation                            | backend/app/raw_python.py:576<br>def raw_python_files(               |
| Export capability selection                             | backend/app/raw_python.py:60<br>def _activity_capabilities(          |
| Regression suite: publisher_reuse                       | backend/tests/test_publisher_reuse.py:1<br>(module)                  |
| Regression suite: raw_python                            | backend/tests/test_raw_python.py:1<br>(module)                       |
| Regression suite: engine_export                         | backend/tests/test_engine_export.py:1<br>(module)                    |

## What to change

When changing Kafka Receive Message, select type=kafka and operation=receive, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of receive. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_publisher_reuse tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Kafka Send Message: source map

Developer guide / Activities / Kafka

## Trace

Saved identity: type=kafka; config.operation=send. Kafka Send Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                       | Source location / search anchor                                      |
|------------------------------------------------------|----------------------------------------------------------------------|
| Activity dispatch                                    | backend/app/runtime.py:694<br>async def execute(self,                |
| Publish outputs and captured inputs                  | backend/app/runtime.py:550<br>def record_activity_output(            |
| Debug stepping over the runtime                      | backend/app/debugger.py:274<br>async def step(                       |
| Contract branch: kafka; select operation send        | frontend/src/ActivityEditor.tsx:783<br>if (n.type === "kafka")       |
| Kafka producer/consumer operation dispatch           | backend/app/runtime.py:1143<br>async def messaging(                  |
| Direct Python implementation; inspect operation send | backend/app/raw_python_support/connectors.py:228<br>async def kafka( |
| Direct graph/code generation                         | backend/app/raw_python.py:576<br>def raw_python_files(               |
| Export capability selection                          | backend/app/raw_python.py:60<br>def _activity_capabilities(          |
| Regression suite: publisher_reuse                    | backend/tests/test_publisher_reuse.py:1<br>(module)                  |
| Regression suite: raw_python                         | backend/tests/test_raw_python.py:1<br>(module)                       |
| Regression suite: engine_export                      | backend/tests/test_engine_export.py:1<br>(module)                    |

## What to change

When changing Kafka Send Message, select type=kafka and operation=send, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of send. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_publisher_reuse tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Kafka Get Messages: source map

Developer guide / Activities / Kafka

## Trace

Saved identity: type=kafka; config.operation=get. Kafka Get Messages is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                      |
|-----------------------------------------------------|----------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,                |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(            |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                       |
| Contract branch: kafka; select operation get        | frontend/src/ActivityEditor.tsx:783<br>if (n.type === "kafka")       |
| Kafka producer/consumer operation dispatch          | backend/app/runtime.py:1143<br>async def messaging(                  |
| Direct Python implementation; inspect operation get | backend/app/raw_python_support/connectors.py:228<br>async def kafka( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(               |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(          |
| Regression suite: publisher_reuse                   | backend/tests/test_publisher_reuse.py:1<br>(module)                  |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                       |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                    |

## What to change

When changing Kafka Get Messages, select type=kafka and operation=get, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of get. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_publisher_reuse tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Pub/Sub Subscriber: source map

Developer guide / Activities / GCP Pub/Sub

## Trace

Saved identity: type=pubsub; config.operation=subscribe. Pub/Sub Subscriber is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                            | Source location / search anchor                                       |
|-----------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                         | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                       | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                           | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: pubsub; select operation subscribe       | frontend/src/ActivityEditor.tsx:810<br>if (n.type === "pubsub")       |
| Pub/Sub publish/receive dispatch                          | backend/app/runtime.py:1143<br>async def messaging(                   |
| Direct Python implementation; inspect operation subscribe | backend/app/raw_python_support/connectors.py:317<br>async def pubsub( |
| Credential/channel client factory                         | backend/app/google_pubsub.py:186<br>def create_client(                |
| Direct graph/code generation                              | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                               | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: google_pubsub                           | backend/tests/test_google_pubsub.py:1<br>(module)                     |
| Regression suite: raw_python                              | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                           | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing Pub/Sub Subscriber, select type=pubsub and operation=subscribe, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of subscribe. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_google_pubsub tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Pub/Sub Publisher: source map

Developer guide / Activities / GCP Pub/Sub

## Trace

Saved identity: type=pubsub; config.operation=publish. Pub/Sub Publisher is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def messaging(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                          | Source location / search anchor                                       |
|---------------------------------------------------------|-----------------------------------------------------------------------|
| Activity dispatch                                       | backend/app/runtime.py:694<br>async def execute(self,                 |
| Publish outputs and captured inputs                     | backend/app/runtime.py:550<br>def record_activity_output(             |
| Debug stepping over the runtime                         | backend/app/debugger.py:274<br>async def step(                        |
| Contract branch: pubsub; select operation publish       | frontend/src/ActivityEditor.tsx:810<br>if (n.type === "pubsub")       |
| Pub/Sub publish/receive dispatch                        | backend/app/runtime.py:1143<br>async def messaging(                   |
| Direct Python implementation; inspect operation publish | backend/app/raw_python_support/connectors.py:317<br>async def pubsub( |
| Credential/channel client factory                       | backend/app/google_pubsub.py:186<br>def create_client(                |
| Direct graph/code generation                            | backend/app/raw_python.py:576<br>def raw_python_files(                |
| Export capability selection                             | backend/app/raw_python.py:60<br>def _activity_capabilities(           |
| Regression suite: google_pubsub                         | backend/tests/test_google_pubsub.py:1<br>(module)                     |
| Regression suite: raw_python                            | backend/tests/test_raw_python.py:1<br>(module)                        |
| Regression suite: engine_export                         | backend/tests/test_engine_export.py:1<br>(module)                     |

## What to change

When changing Pub/Sub Publisher, select type=pubsub and operation=publish, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of publish. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_google_pubsub tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Dynamic Connection: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=dynamic\_connection. Dynamic Connection is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                        | Source location / search anchor                                    |
|-----------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                     | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                                   | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                       | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation<br>dynamic_connection          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use<br>receive_idoc                 | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect<br>operation dynamic_connection | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                        | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                                  | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                          | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                           | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                          | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                          | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                       | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing Dynamic Connection, select type=sap and operation=dynamic\_connection, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of dynamic\_connection. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Acknowledgment: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_acknowledgment. IDoc Acknowledgment is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                      | Source location / search anchor                                    |
|---------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                   | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                                 | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                     | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_acknowledgment          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc                  | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_acknowledgment | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                      | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                                | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                        | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                         | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                        | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                        | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                     | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Acknowledgment, select type=sap and operation=idoc\_acknowledgment, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of idoc\_acknowledgment. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Confirmation: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_confirmation. IDoc Confirmation is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                    | Source location / search anchor                                    |
|-------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                 | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                               | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                   | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_confirmation          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc                | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_confirmation | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                    | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                              | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                      | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                       | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                      | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                      | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                   | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Confirmation, select type=sap and operation=idoc\_confirmation, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of idoc\_confirmation. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Converter: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_converter. IDoc Converter is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                 | Source location / search anchor                                    |
|----------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                              | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                            | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_converter          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc             | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_converter | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                 | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                           | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                   | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                    | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                   | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                   | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Converter, select type=sap and operation=idoc\_converter, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of idoc\_converter. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Listener: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_listener. IDoc Listener is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                | Source location / search anchor                                    |
|---------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                             | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                           | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                               | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_listener          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc            | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_listener | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Inbound SAP adapter                                           | backend/app/sap.py:793<br>async def receive_idoc(                  |
| Continuous SAP jobs                                           | backend/app/main.py:438<br>async def _continuous_sap_event_loop(   |
| Python/native worker lifecycle                                | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                          | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                  | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                   | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                  | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                  | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                               | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Listener, select type=sap and operation=idoc\_listener, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of `idoc_listener`. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Parser: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_parser. IDoc Parser is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                              | Source location / search anchor                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                           | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                         | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                             | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_parser          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc          | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_parser | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                              | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                        | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                 | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                             | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Parser, select type=sap and operation=idoc\_parser, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of idoc\_parser. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Reader: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_reader. IDoc Reader is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                              | Source location / search anchor                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                           | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                         | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                             | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_reader          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc          | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_reader | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                              | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                        | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                 | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                             | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Reader, select type=sap and operation=idoc\_reader, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of idoc\_reader. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Post IDoc to SAP: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=post\_idoc. Post IDoc to SAP is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                            | Source location / search anchor                                    |
|-----------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                         | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                       | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                           | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation post_idoc          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc        | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation post_idoc | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                            | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                      | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                              | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                               | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                              | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                              | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                           | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing Post IDoc to SAP, select type=sap and operation=post\_idoc, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of post\_idoc. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# IDoc Renderer: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=idoc\_renderer. IDoc Renderer is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                | Source location / search anchor                                    |
|---------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                             | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                           | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                               | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation idoc_renderer          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc            | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation idoc_renderer | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                          | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                  | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                   | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                  | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                  | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                               | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing IDoc Renderer, select type=sap and operation=idoc\_renderer, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of idoc\_renderer. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# RFC BAPI Listener: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=rfc\_bapi\_listener. RFC BAPI Listener is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                       | Source location / search anchor                                    |
|----------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                    | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                                  | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                      | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation<br>rfc_bapi_listener          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use<br>receive_idoc                | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect<br>operation rfc_bapi_listener | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Inbound SAP adapter                                                  | backend/app/sap.py:793<br>async def receive_idoc(                  |
| Continuous SAP jobs                                                  | backend/app/main.py:438<br>async def _continuous_sap_event_loop(   |
| Python/native worker lifecycle                                       | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                                 | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                         | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                          | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                         | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                         | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                      | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing RFC BAPI Listener, select type=sap and operation=rfc\_bapi\_listener, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of `rfc_bapi_listener`. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Invoke RFC BAPI in SAP: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=invoke\_rfc\_bapi. Invoke RFC BAPI in SAP is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                     | Source location / search anchor                                    |
|--------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                  | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                                | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                    | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation<br>invoke_rfc_bapi          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use<br>receive_idoc              | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect<br>operation invoke_rfc_bapi | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                     | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                               | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                       | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                        | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                       | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                       | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                    | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing Invoke RFC BAPI in SAP, select type=sap and operation=invoke\_rfc\_bapi, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of invoke\_rfc\_bapi. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Reply from RFC BAPI in SAP: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=reply\_rfc\_bapi. Reply from RFC BAPI in SAP is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                    | Source location / search anchor                                    |
|-------------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                                 | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                               | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                                   | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation<br>reply_rfc_bapi          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use<br>receive_idoc             | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect<br>operation reply_rfc_bapi | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                                    | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                              | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                                      | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                       | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                                      | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                                      | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                                   | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing Reply from RFC BAPI in SAP, select type=sap and operation=reply\_rfc\_bapi, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of reply\_rfc\_bapi. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Read Table: source map

Developer guide / Activities / SAP ECC

## Trace

Saved identity: type=sap; config.operation=read\_table. Read Table is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/sap.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                             | Source location / search anchor                                    |
|------------------------------------------------------------|--------------------------------------------------------------------|
| Activity dispatch                                          | backend/app/runtime.py:694<br>async def execute(self,              |
| Publish outputs and captured inputs                        | backend/app/runtime.py:550<br>def record_activity_output(          |
| Debug stepping over the runtime                            | backend/app/debugger.py:274<br>async def step(                     |
| Contract branch: sap; select operation read_table          | frontend/src/ActivityEditor.tsx:820<br>if (n.type === "sap")       |
| SAP operation dispatch; listeners use receive_idoc         | backend/app/sap.py:1081<br>def execute(                            |
| Direct Python implementation; inspect operation read_table | backend/app/raw_python_support/connectors.py:197<br>async def sap( |
| Python/native worker lifecycle                             | backend/app/java_bridge.py:1<br>(module)                           |
| Native Java connector implementation                       | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)  |
| Direct graph/code generation                               | backend/app/raw_python.py:576<br>def raw_python_files(             |
| Export capability selection                                | backend/app/raw_python.py:60<br>def _activity_capabilities(        |
| Regression suite: sap_mapper                               | backend/tests/test_sap_mapper.py:1<br>(module)                     |
| Regression suite: raw_python                               | backend/tests/test_raw_python.py:1<br>(module)                     |
| Regression suite: engine_export                            | backend/tests/test_engine_export.py:1<br>(module)                  |

## What to change

When changing Read Table, select type=sap and operation=read\_table, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of read\_table. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_sap_mapper tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# JDBC Create: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=insert. JDBC Create is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                     |
|--------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation insert         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied           | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation insert | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command              | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing JDBC Create, select type=jdbc and operation=insert, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of insert. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# JDBC Update: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=update. JDBC Update is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                     |
|--------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation update         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied           | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation update | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command              | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing JDBC Update, select type=jdbc and operation=update, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of update. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# JDBC Query: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=query. JDBC Query is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                        | Source location / search anchor                                     |
|-------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                     | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                   | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                       | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation query         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied          | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation query | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                          | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                           | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command             | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                          | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                       | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing JDBC Query, select type=jdbc and operation=query, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of query. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# JDBC Truncate: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=truncate. JDBC Truncate is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                           | Source location / search anchor                                     |
|----------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                        | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                      | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                          | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation truncate         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied             | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation truncate | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                             | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                              | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command                | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                             | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                          | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing JDBC Truncate, select type=jdbc and operation=truncate, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of truncate. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# JDBC Delete: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=delete. JDBC Delete is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                     |
|--------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation delete         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied           | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation delete | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command              | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing JDBC Delete, select type=jdbc and operation=delete, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of delete. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Stored Procedure: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=call. Stored Procedure is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                       | Source location / search anchor                                     |
|------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                    | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                  | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                      | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation call         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied         | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation call | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                         | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                          | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command            | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                         | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                      | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing Stored Procedure, select type=jdbc and operation=call, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of call. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Dynamic SQL: source map

Developer guide / Activities / JDBC

## Trace

Saved identity: type=jdbc; config.operation=dynamic. Dynamic SQL is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/jdbc.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                          | Source location / search anchor                                     |
|---------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                       | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                     | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                         | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: jdbc; select operation dynamic         | frontend/src/ActivityEditor.tsx:605<br>if (n.type === "jdbc")       |
| JDBC adapter; uses transaction when supplied            | backend/app/jdbc.py:277<br>def execute(                             |
| Direct Python implementation; inspect operation dynamic | backend/app/raw_python_support/connectors.py:154<br>async def jdbc( |
| Direct graph/code generation                            | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                             | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command               | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                            | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                         | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing Dynamic SQL, select type=jdbc and operation=dynamic, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of dynamic. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Snowflake Insert: source map

Developer guide / Activities / Snowflake

## Trace

Saved identity: type=snowflake; config.operation=insert. Snowflake Insert is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/snowflake.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                          |
|--------------------------------------------------------|--------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                    |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(                |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                           |
| Contract branch: snowflake; select operation insert    | frontend/src/ActivityEditor.tsx:638<br>if (n.type === "snowflake")       |
| Snowflake operation adapter                            | backend/app/snowflake.py:231<br>def execute(                             |
| Direct Python implementation; inspect operation insert | backend/app/raw_python_support/connectors.py:159<br>async def snowflake( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                   |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(              |
| Regression suite: snowflake                            | backend/tests/test_snowflake.py:1<br>(module)                            |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                           |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                        |

## What to change

When changing Snowflake Insert, select type=snowflake and operation=insert, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of insert. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_snowflake tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Snowflake Query: source map

Developer guide / Activities / Snowflake

## Trace

Saved identity: type=snowflake; config.operation=query. Snowflake Query is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/snowflake.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                        | Source location / search anchor                                          |
|-------------------------------------------------------|--------------------------------------------------------------------------|
| Activity dispatch                                     | backend/app/runtime.py:694<br>async def execute(self,                    |
| Publish outputs and captured inputs                   | backend/app/runtime.py:550<br>def record_activity_output(                |
| Debug stepping over the runtime                       | backend/app/debugger.py:274<br>async def step(                           |
| Contract branch: snowflake; select operation query    | frontend/src/ActivityEditor.tsx:638<br>if (n.type === "snowflake")       |
| Snowflake operation adapter                           | backend/app/snowflake.py:231<br>def execute(                             |
| Direct Python implementation; inspect operation query | backend/app/raw_python_support/connectors.py:159<br>async def snowflake( |
| Direct graph/code generation                          | backend/app/raw_python.py:576<br>def raw_python_files(                   |
| Export capability selection                           | backend/app/raw_python.py:60<br>def _activity_capabilities(              |
| Regression suite: snowflake                           | backend/tests/test_snowflake.py:1<br>(module)                            |
| Regression suite: raw_python                          | backend/tests/test_raw_python.py:1<br>(module)                           |
| Regression suite: engine_export                       | backend/tests/test_engine_export.py:1<br>(module)                        |

## What to change

When changing Snowflake Query, select type=snowflake and operation=query, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of query. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_snowflake tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Snowflake Update: source map

Developer guide / Activities / Snowflake

## Trace

Saved identity: type=snowflake; config.operation=update. Snowflake Update is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/snowflake.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                          |
|--------------------------------------------------------|--------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                    |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(                |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                           |
| Contract branch: snowflake; select operation update    | frontend/src/ActivityEditor.tsx:638<br>if (n.type === "snowflake")       |
| Snowflake operation adapter                            | backend/app/snowflake.py:231<br>def execute(                             |
| Direct Python implementation; inspect operation update | backend/app/raw_python_support/connectors.py:159<br>async def snowflake( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                   |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(              |
| Regression suite: snowflake                            | backend/tests/test_snowflake.py:1<br>(module)                            |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                           |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                        |

## What to change

When changing Snowflake Update, select type=snowflake and operation=update, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of update. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_snowflake tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Snowflake Delete: source map

Developer guide / Activities / Snowflake

## Trace

Saved identity: type=snowflake; config.operation=delete. Snowflake Delete is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/snowflake.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                          |
|--------------------------------------------------------|--------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                    |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(                |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                           |
| Contract branch: snowflake; select operation delete    | frontend/src/ActivityEditor.tsx:638<br>if (n.type === "snowflake")       |
| Snowflake operation adapter                            | backend/app/snowflake.py:231<br>def execute(                             |
| Direct Python implementation; inspect operation delete | backend/app/raw_python_support/connectors.py:159<br>async def snowflake( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                   |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(              |
| Regression suite: snowflake                            | backend/tests/test_snowflake.py:1<br>(module)                            |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                           |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                        |

## What to change

When changing Snowflake Delete, select type=snowflake and operation=delete, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of delete. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_snowflake tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Snowflake Bulk Load: source map

Developer guide / Activities / Snowflake

## Trace

Saved identity: type=snowflake; config.operation=bulk\_load. Snowflake Bulk Load is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/snowflake.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                            | Source location / search anchor                                          |
|-----------------------------------------------------------|--------------------------------------------------------------------------|
| Activity dispatch                                         | backend/app/runtime.py:694<br>async def execute(self,                    |
| Publish outputs and captured inputs                       | backend/app/runtime.py:550<br>def record_activity_output(                |
| Debug stepping over the runtime                           | backend/app/debugger.py:274<br>async def step(                           |
| Contract branch: snowflake; select operation bulk_load    | frontend/src/ActivityEditor.tsx:638<br>if (n.type === "snowflake")       |
| Snowflake operation adapter                               | backend/app/snowflake.py:231<br>def execute(                             |
| Direct Python implementation; inspect operation bulk_load | backend/app/raw_python_support/connectors.py:159<br>async def snowflake( |
| Direct graph/code generation                              | backend/app/raw_python.py:576<br>def raw_python_files(                   |
| Export capability selection                               | backend/app/raw_python.py:60<br>def _activity_capabilities(              |
| Regression suite: snowflake                               | backend/tests/test_snowflake.py:1<br>(module)                            |
| Regression suite: raw_python                              | backend/tests/test_raw_python.py:1<br>(module)                           |
| Regression suite: engine_export                           | backend/tests/test_engine_export.py:1<br>(module)                        |

## What to change

When changing Snowflake Bulk Load, select type=snowflake and operation=bulk\_load, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of bulk\_load. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_snowflake tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# AMQP Receive Message: source map

Developer guide / Activities / AMQP

## Trace

Saved identity: type=amqp; config.operation=receive. AMQP Receive Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/amqp.py :: def send(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                          | Source location / search anchor                                     |
|---------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                       | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                     | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                         | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: amqp; select operation receive         | frontend/src/ActivityEditor.tsx:629<br>if (n.type === "amqp")       |
| AMQP send adapter; get handles receives                 | backend/app/amqp.py:135<br>def send(                                |
| Direct Python implementation; inspect operation receive | backend/app/raw_python_support/connectors.py:164<br>async def amqp( |
| Receive adapter and acknowledgement callback            | backend/app/amqp.py:182<br>def get(                                 |
| Direct graph/code generation                            | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                             | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command               | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                            | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                         | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing AMQP Receive Message, select type=amqp and operation=receive, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of receive. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# AMQP Get Message: source map

Developer guide / Activities / AMQP

## Trace

Saved identity: type=amqp; config.operation=get. AMQP Get Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/amqp.py :: def send(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                     |
|-----------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: amqp; select operation get         | frontend/src/ActivityEditor.tsx:629<br>if (n.type === "amqp")       |
| AMQP send adapter; get handles receives             | backend/app/amqp.py:135<br>def send(                                |
| Direct Python implementation; inspect operation get | backend/app/raw_python_support/connectors.py:164<br>async def amqp( |
| Receive adapter and acknowledgement callback        | backend/app/amqp.py:182<br>def get(                                 |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command           | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing AMQP Get Message, select type=amqp and operation=get, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of get. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# AMQP Send Message: source map

Developer guide / Activities / AMQP

## Trace

Saved identity: type=amqp; config.operation=send. AMQP Send Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/amqp.py :: def send(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                       | Source location / search anchor                                     |
|------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                    | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                  | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                      | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: amqp; select operation send         | frontend/src/ActivityEditor.tsx:629<br>if (n.type === "amqp")       |
| AMQP send adapter; get handles receives              | backend/app/amqp.py:135<br>def send(                                |
| Direct Python implementation; inspect operation send | backend/app/raw_python_support/connectors.py:164<br>async def amqp( |
| Direct graph/code generation                         | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                          | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command            | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                         | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                      | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing AMQP Send Message, select type=amqp and operation=send, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of send. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# AMQP Dead Letter Message: source map

Developer guide / Activities / AMQP

## Trace

Saved identity: type=amqp; config.operation=dead\_letter. AMQP Dead Letter Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/amqp.py :: def send(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                 | Source location / search anchor                                     |
|----------------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                              | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                            | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                                | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: amqp; select operation<br>dead_letter         | frontend/src/ActivityEditor.tsx:629<br>if (n.type === "amqp")       |
| AMQP send adapter; get handles receives                        | backend/app/amqp.py:135<br>def send(                                |
| Direct Python implementation; inspect<br>operation dead_letter | backend/app/raw_python_support/connectors.py:164<br>async def amqp( |
| Receive adapter and acknowledgement<br>callback                | backend/app/amqp.py:182<br>def get(                                 |
| Direct graph/code generation                                   | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                                    | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command                      | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                                   | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                                | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing AMQP Dead Letter Message, select type=amqp and operation=dead\_letter, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of dead\_letter. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment.

Do not patch only an already-generated archive.

# Parse XML: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=xml; config.operation=parse. Parse XML is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def parse_xml_activity(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                        | Source location / search anchor                                        |
|-------------------------------------------------------|------------------------------------------------------------------------|
| Activity dispatch                                     | backend/app/runtime.py:694<br>async def execute(self,                  |
| Publish outputs and captured inputs                   | backend/app/runtime.py:550<br>def record_activity_output(              |
| Debug stepping over the runtime                       | backend/app/debugger.py:274<br>async def step(                         |
| XML parse; render_xml_activity handles render         | backend/app/runtime.py:1840<br>def parse_xml_activity(                 |
| Direct Python implementation; inspect operation parse | backend/app/raw_python_support/activities.py:134<br>def data_activity( |
| Direct graph/code generation                          | backend/app/raw_python.py:576<br>def raw_python_files(                 |
| Export capability selection                           | backend/app/raw_python.py:60<br>def _activity_capabilities(            |
| Regression suite: activity_packs                      | backend/tests/test_activity_packs.py:1<br>(module)                     |
| Regression suite: raw_python                          | backend/tests/test_raw_python.py:1<br>(module)                         |
| Regression suite: engine_export                       | backend/tests/test_engine_export.py:1<br>(module)                      |

## What to change

When changing Parse XML, select type=xml and operation=parse, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of parse. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Render XML: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=xml; config.operation=render. Render XML is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def parse_xml_activity(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                        |
|--------------------------------------------------------|------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                  |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(              |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                         |
| XML parse; render_xml_activity handles render          | backend/app/runtime.py:1840<br>def parse_xml_activity(                 |
| Direct Python implementation; inspect operation render | backend/app/raw_python_support/activities.py:134<br>def data_activity( |
| XML render implementation                              | backend/app/runtime.py:1856<br>def render_xml_activity(                |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                 |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(            |
| Regression suite: activity_packs                       | backend/tests/test_activity_packs.py:1<br>(module)                     |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                         |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                      |

## What to change

When changing Render XML, select type=xml and operation=render, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of render. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Parse JSON: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=json; config.operation=parse. Parse JSON is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def json_activity(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                        | Source location / search anchor                                        |
|-------------------------------------------------------|------------------------------------------------------------------------|
| Activity dispatch                                     | backend/app/runtime.py:694<br>async def execute(self,                  |
| Publish outputs and captured inputs                   | backend/app/runtime.py:550<br>def record_activity_output(              |
| Debug stepping over the runtime                       | backend/app/debugger.py:274<br>async def step(                         |
| JSON parse/render and validation                      | backend/app/runtime.py:1928<br>def json_activity(                      |
| Direct Python implementation; inspect operation parse | backend/app/raw_python_support/activities.py:134<br>def data_activity( |
| Direct graph/code generation                          | backend/app/raw_python.py:576<br>def raw_python_files(                 |
| Export capability selection                           | backend/app/raw_python.py:60<br>def _activity_capabilities(            |
| Regression suite: activity_packs                      | backend/tests/test_activity_packs.py:1<br>(module)                     |
| Regression suite: raw_python                          | backend/tests/test_raw_python.py:1<br>(module)                         |
| Regression suite: engine_export                       | backend/tests/test_engine_export.py:1<br>(module)                      |

## What to change

When changing Parse JSON, select type=json and operation=parse, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of parse. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Render JSON: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=json; config.operation=render. Render JSON is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def json_activity(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                        |
|--------------------------------------------------------|------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                  |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(              |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                         |
| JSON parse/render and validation                       | backend/app/runtime.py:1928<br>def json_activity(                      |
| Direct Python implementation; inspect operation render | backend/app/raw_python_support/activities.py:134<br>def data_activity( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                 |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(            |
| Regression suite: activity_packs                       | backend/tests/test_activity_packs.py:1<br>(module)                     |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                         |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                      |

## What to change

When changing Render JSON, select type=json and operation=render, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of render. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Parse Data: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=flat; config.operation=parse. Parse Data is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def flat_data(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                        | Source location / search anchor                                        |
|-------------------------------------------------------|------------------------------------------------------------------------|
| Activity dispatch                                     | backend/app/runtime.py:694<br>async def execute(self,                  |
| Publish outputs and captured inputs                   | backend/app/runtime.py:550<br>def record_activity_output(              |
| Debug stepping over the runtime                       | backend/app/debugger.py:274<br>async def step(                         |
| Flat record parse/render                              | backend/app/runtime.py:1968<br>def flat_data(                          |
| Direct Python implementation; inspect operation parse | backend/app/raw_python_support/activities.py:134<br>def data_activity( |
| Direct graph/code generation                          | backend/app/raw_python.py:576<br>def raw_python_files(                 |
| Export capability selection                           | backend/app/raw_python.py:60<br>def _activity_capabilities(            |
| Regression suite: activity_packs                      | backend/tests/test_activity_packs.py:1<br>(module)                     |
| Regression suite: raw_python                          | backend/tests/test_raw_python.py:1<br>(module)                         |
| Regression suite: engine_export                       | backend/tests/test_engine_export.py:1<br>(module)                      |

## What to change

When changing Parse Data, select type=flat and operation=parse, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of parse. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Render Data: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=flat; config.operation=render. Render Data is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def flat_data(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                        |
|--------------------------------------------------------|------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                  |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(              |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                         |
| Flat record parse/render                               | backend/app/runtime.py:1968<br>def flat_data(                          |
| Direct Python implementation; inspect operation render | backend/app/raw_python_support/activities.py:134<br>def data_activity( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                 |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(            |
| Regression suite: activity_packs                       | backend/tests/test_activity_packs.py:1<br>(module)                     |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                         |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                      |

## What to change

When changing Render Data, select type=flat and operation=render, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of render. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_activity_packs tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Read Excel Workbook: source map

Developer guide / Activities / Data

## Trace

Saved identity: type=excel; config.operation=read. Read Excel Workbook is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: def read_excel(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                       | Source location / search anchor                                     |
|------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                    | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                  | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                      | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: excel; select operation read        | frontend/src/ActivityEditor.tsx:680<br>if (n.type === "excel")      |
| Workbook reading                                     | backend/app/runtime.py:1687<br>def read_excel(                      |
| Direct Python implementation; inspect operation read | backend/app/raw_python_support/activities.py:185<br>def excel_read( |
| Direct graph/code generation                         | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                          | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: amqp_jdbc_excel_command            | backend/tests/test_amqp_jdbc_excel_command.py:1<br>(module)         |
| Regression suite: raw_python                         | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                      | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing Read Excel Workbook, select type=excel and operation=read, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of read. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_amqp_jdbc_excel_command tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Mapper: source map

Developer guide / Activities / General

## Trace

Saved identity: type=mapper; config.operation=map. Mapper is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/mapper.py :: def execute(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                      | Source location / search anchor                                  |
|-----------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                   | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                 | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                     | backend/app/debugger.py:274<br>async def step(                   |
| Schema mapping engine                               | backend/app/mapper.py:410<br>def execute(                        |
| Direct Python implementation; inspect operation map | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                        | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                         | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general               | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                     | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Mapper, select type=mapper and operation=map, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of map. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Transform: source map

Developer guide / Activities / General

## Trace

Saved identity: type=dataweave; config.operation=transform. Transform is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/dataweave.py :: def execute_details(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                            | Source location / search anchor                                     |
|-----------------------------------------------------------|---------------------------------------------------------------------|
| Activity dispatch                                         | backend/app/runtime.py:694<br>async def execute(self,               |
| Publish outputs and captured inputs                       | backend/app/runtime.py:550<br>def record_activity_output(           |
| Debug stepping over the runtime                           | backend/app/debugger.py:274<br>async def step(                      |
| Contract branch: dataweave; select operation transform    | frontend/src/ActivityEditor.tsx:1084<br>if (n.type === "dataweave") |
| Transformation script parser and evaluator                | backend/app/dataweave.py:346<br>def execute_details(                |
| Direct Python implementation; inspect operation transform | backend/app/raw_python_support/core.py:182<br>async def execute(    |
| Direct graph/code generation                              | backend/app/raw_python.py:576<br>def raw_python_files(              |
| Export capability selection                               | backend/app/raw_python.py:60<br>def _activity_capabilities(         |
| Regression suite: mapper_code_general                     | backend/tests/test_mapper_code_general.py:1<br>(module)             |
| Regression suite: raw_python                              | backend/tests/test_raw_python.py:1<br>(module)                      |
| Regression suite: engine_export                           | backend/tests/test_engine_export.py:1<br>(module)                   |

## What to change

When changing Transform, select type=dataweave and operation=transform, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of transform. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Log: source map

Developer guide / Activities / General

## Trace

Saved identity: type=log; config.operation=write. Log is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / Debugger
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (log)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                          | Source location / search anchor                                  |
|-------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                       | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                     | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                         | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: log; select operation write                            | frontend/src/ActivityEditor.tsx:1095<br>if (n.type === "log")    |
| Engine log dispatch; operation write                                    | backend/app/runtime.py:816<br>if activity.type == 'log':         |
| Direct helper dispatch for log; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                            | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                             | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                         | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                            | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                         | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Log, select type=log and operation=write, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of write. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Catch Exception: source map

Developer guide / Activities / General

## Trace

Saved identity: type=catch; config.operation=catch. Catch Exception is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (catch)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: catch; select operation catch                            | frontend/src/ActivityEditor.tsx:282<br>if (n.type === "catch")   |
| Engine catch dispatch; operation catch                                    | backend/app/runtime.py:798<br>if activity.type == 'catch':       |
| Direct helper dispatch for catch; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                           | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Catch Exception, select type=catch and operation=catch, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of catch. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Throw Exception: source map

Developer guide / Activities / General

## Trace

Saved identity: type=throw; config.operation=throw. Throw Exception is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (throw)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: throw; select operation throw                            | frontend/src/ActivityEditor.tsx:289<br>if (n.type === "throw")   |
| Engine throw dispatch; operation throw                                    | backend/app/runtime.py:799<br>if activity.type == 'throw':       |
| Direct helper dispatch for throw; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                           | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Throw Exception, select type=throw and operation=throw, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of throw. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Rethrow Exception: source map

Developer guide / Activities / General

## Trace

Saved identity: type=rethrow; config.operation=rethrow. Rethrow Exception is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (rethrow)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                              | Source location / search anchor                                  |
|-----------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                           | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                         | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                             | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: rethrow; select operation rethrow                          | frontend/src/ActivityEditor.tsx:296<br>if (n.type === "rethrow") |
| Engine rethrow dispatch; operation rethrow                                  | backend/app/runtime.py:807<br>if activity.type == 'rethrow':     |
| Direct helper dispatch for rethrow; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                                | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                                 | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                             | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                                | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                             | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Rethrow Exception, select type=rethrow and operation=rethrow, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of rethrow. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Confirm Message: source map

Developer guide / Activities / General

## Trace

Saved identity: type=confirm; config.operation=acknowledge. Confirm Message is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (confirm)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                              | Source location / search anchor                                  |
|-----------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                           | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                         | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                             | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: confirm; select operation acknowledge                      | frontend/src/ActivityEditor.tsx:268<br>if (n.type === "confirm") |
| Engine confirm dispatch; operation acknowledge                              | backend/app/runtime.py:849<br>if activity.type == 'confirm':     |
| Direct helper dispatch for confirm; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                                | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                                 | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: tasks_runtime                                             | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: raw_python                                                | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                             | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Confirm Message, select type=confirm and operation=acknowledge, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of acknowledge. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_tasks_runtime tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Java Activity: source map

Developer guide / Activities / General

## Trace

Saved identity: type=java; config.operation=invoke. Java Activity is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def java_worker(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                            |
|--------------------------------------------------------|----------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                      |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(                  |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                             |
| Contract branch: java; select operation invoke         | frontend/src/ActivityEditor.tsx:1117<br>if (n.type === "java")             |
| Java invocation worker                                 | backend/app/runtime.py:2073<br>async def java_worker(                      |
| Direct Python implementation; inspect operation invoke | backend/app/raw_python_support/activities.py:323<br>async def java_invoke( |
| Python/native worker lifecycle                         | backend/app/java_bridge.py:1<br>(module)                                   |
| Native Java connector implementation                   | java-bridge/src/com/mina/bridge/MinaJavaBridge.java:1<br>(module)          |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                     |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(                |
| Regression suite: mapper_code_general                  | backend/tests/test_mapper_code_general.py:1<br>(module)                    |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                             |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                          |

## What to change

When changing Java Activity, select type=java and operation=invoke, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of invoke. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes `raw_python_support`; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Python Invoke: source map

Developer guide / Activities / General

## Trace

Saved identity: type=python; config.operation=invoke. Python Invoke is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: async def python_worker(
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                         | Source location / search anchor                                              |
|--------------------------------------------------------|------------------------------------------------------------------------------|
| Activity dispatch                                      | backend/app/runtime.py:694<br>async def execute(self,                        |
| Publish outputs and captured inputs                    | backend/app/runtime.py:550<br>def record_activity_output(                    |
| Debug stepping over the runtime                        | backend/app/debugger.py:274<br>async def step(                               |
| Contract branch: python; select operation invoke       | frontend/src/ActivityEditor.tsx:1145<br>if (n.type === "python")             |
| Python invocation worker                               | backend/app/runtime.py:2128<br>async def python_worker(                      |
| Direct Python implementation; inspect operation invoke | backend/app/raw_python_support/activities.py:299<br>async def python_invoke( |
| Direct graph/code generation                           | backend/app/raw_python.py:576<br>def raw_python_files(                       |
| Export capability selection                            | backend/app/raw_python.py:60<br>def _activity_capabilities(                  |
| Regression suite: mapper_code_general                  | backend/tests/test_mapper_code_general.py:1<br>(module)                      |
| Regression suite: raw_python                           | backend/tests/test_raw_python.py:1<br>(module)                               |
| Regression suite: engine_export                        | backend/tests/test_engine_export.py:1<br>(module)                            |

## What to change

When changing Python Invoke, select type=python and operation=invoke, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of invoke. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# External Command: source map

Developer guide / Activities / General

## Trace

Saved identity: type=basic; config.operation=external\_command. External Command is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (basic)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                               | Source location / search anchor                                  |
|------------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                            | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                          | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                              | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: basic; select operation<br>external_command                 | frontend/src/ActivityEditor.tsx:1156<br>if (n.type === "basic")  |
| Engine basic dispatch; operation<br>external_command                         | backend/app/runtime.py:732<br>if activity.type == 'basic':       |
| Direct helper dispatch for basic; task graph<br>may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                                 | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                                  | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general                                        | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                                                 | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                              | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing External Command, select type=basic and operation=external\_command, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of external\_command. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Assign: source map

Developer guide / Activities / General

## Trace

Saved identity: type=basic; config.operation=assign. Assign is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (basic)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: basic; select operation assign                           | frontend/src/ActivityEditor.tsx:1156<br>if (n.type === "basic")  |
| Engine basic dispatch; operation assign                                   | backend/app/runtime.py:732<br>if activity.type == 'basic':       |
| Direct helper dispatch for basic; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general                                     | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Assign, select type=basic and operation=assign, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of assign. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Checkpoint: source map

Developer guide / Activities / General

## Trace

Saved identity: type=basic; config.operation=checkpoint. Checkpoint is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (basic)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: basic; select operation checkpoint                       | frontend/src/ActivityEditor.tsx:1156<br>if (n.type === "basic")  |
| Engine basic dispatch; operation checkpoint                               | backend/app/runtime.py:732<br>if activity.type == 'basic':       |
| Direct helper dispatch for basic; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general                                     | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Checkpoint, select type=basic and operation=checkpoint, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of checkpoint. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Sleep: source map

Developer guide / Activities / General

## Trace

Saved identity: type=basic; config.operation=sleep. Sleep is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / Debugger
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (basic)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                            | Source location / search anchor                                  |
|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                         | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                       | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                           | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: basic; select operation sleep                            | frontend/src/ActivityEditor.tsx:1156<br>if (n.type === "basic")  |
| Engine basic dispatch; operation sleep                                    | backend/app/runtime.py:732<br>if activity.type == 'basic':       |
| Direct helper dispatch for basic; task graph may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                              | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                               | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general                                     | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                                              | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                           | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Sleep, select type=basic and operation=sleep, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of sleep. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Get Shared Variable: source map

Developer guide / Activities / General

## Trace

Saved identity: type=basic; config.operation=get\_shared\_variable. Get Shared Variable is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (basic)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                               | Source location / search anchor                                  |
|------------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                            | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                          | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                              | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: basic; select operation<br>get_shared_variable              | frontend/src/ActivityEditor.tsx:1156<br>if (n.type === "basic")  |
| Engine basic dispatch; operation<br>get_shared_variable                      | backend/app/runtime.py:732<br>if activity.type == 'basic':       |
| Direct helper dispatch for basic; task graph<br>may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                                 | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                                  | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general                                        | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                                                 | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                              | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Get Shared Variable, select type=basic and operation=get\_shared\_variable, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of get\_shared\_variable. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Set Shared Variable: source map

Developer guide / Activities / General

## Trace

Saved identity: type=basic; config.operation=set\_shared\_variable. Set Shared Variable is a palette definition rendered by shared React components, not necessarily its own frontend file or Python module.

```
main.tsx packs -> addActivity -> ActivityEditor
Project.tasks[].activities[] -> Run API / DebugManager
WorkflowRuntime.execute_with_policy -> execute
backend/app/runtime.py :: execute (basic)
record_activity_output -> downstream mappings / Job Data
```

For event starters the API/listener loop supplies event\_output before the graph continues; a manual single-activity test is not equivalent to the continuous listener lifecycle.

## Source locations

| Responsibility                                  | Source location / search anchor                                            |
|-------------------------------------------------|----------------------------------------------------------------------------|
| Palette labels, types, operations and icons     | frontend/src/main.tsx:332<br>const packs:                                  |
| Create a canvas activity and its initial config | frontend/src/main.tsx:1770<br>const addActivity =                          |
| Configuration/Input/Output/Errors contract      | frontend/src/ActivityEditor.tsx:191<br>export function activityContract(   |
| Project-specific schema resolution              | frontend/src/ActivityEditor.tsx:1197<br>function resolvedActivityContract( |
| Configuration keys exposed for mapping          | frontend/src/ActivityEditor.tsx:1273<br>function runtimeMappableInputs(    |
| Shared configuration field renderer             | frontend/src/ActivityEditor.tsx:1721<br>function FieldEditor(              |
| Input mappings editor                           | frontend/src/ActivityEditor.tsx:2115<br>function InputEditor(              |
| Output schema editor                            | frontend/src/ActivityEditor.tsx:2238<br>function OutputEditor(             |
| Advanced logging/retry UI                       | frontend/src/ActivityEditor.tsx:2290<br>function AdvancedEditor(           |
| Error handling UI                               | frontend/src/ActivityEditor.tsx:2446<br>function ErrorEditor(              |
| Persisted activity model                        | backend/app/models.py:17<br>class Activity(                                |
| Run HTTP API                                    | backend/app/main.py:616<br>@app.post('/api/projects/{project_id}/run')     |
| Graph scheduler                                 | backend/app/runtime.py:355<br>async def run(self,                          |
| Retry/logging policy wrapper                    | backend/app/runtime.py:601<br>async def execute_with_policy(               |
| Resolve configuration and input mappings        | backend/app/runtime.py:670<br>def resolve_activity_config(                 |

| Responsibility                                                               | Source location / search anchor                                  |
|------------------------------------------------------------------------------|------------------------------------------------------------------|
| Activity dispatch                                                            | backend/app/runtime.py:694<br>async def execute(self,            |
| Publish outputs and captured inputs                                          | backend/app/runtime.py:550<br>def record_activity_output(        |
| Debug stepping over the runtime                                              | backend/app/debugger.py:274<br>async def step(                   |
| Contract branch: basic; select operation<br>set_shared_variable              | frontend/src/ActivityEditor.tsx:1156<br>if (n.type === "basic")  |
| Engine basic dispatch; operation<br>set_shared_variable                      | backend/app/runtime.py:732<br>if activity.type == 'basic':       |
| Direct helper dispatch for basic; task graph<br>may also be generated inline | backend/app/raw_python_support/core.py:182<br>async def execute( |
| Direct graph/code generation                                                 | backend/app/raw_python.py:576<br>def raw_python_files(           |
| Export capability selection                                                  | backend/app/raw_python.py:60<br>def _activity_capabilities(      |
| Regression suite: mapper_code_general                                        | backend/tests/test_mapper_code_general.py:1<br>(module)          |
| Regression suite: raw_python                                                 | backend/tests/test_raw_python.py:1<br>(module)                   |
| Regression suite: engine_export                                              | backend/tests/test_engine_export.py:1<br>(module)                |

## What to change

When changing Set Shared Variable, select type=basic and operation=set\_shared\_variable, not merely the displayed label. The shared dispatcher may serve several activities. Add/adjust editor fields, resolved input values and returned output keys together; preserve behavior of the other operations sharing this handler.

Configuration tab: activityContract and FieldEditor. Input tab: runtimeMappableInputs/InputEditor; mappings persist as config.inputMappings and are resolved before execute. Output tab: contract/resolved schema plus actual returned keys. Advanced tab: advancedDefaults/AdvancedEditor and execute\_with\_policy. Errors tab: declared faults/ErrorEditor plus real exceptions and run/debug propagation. UI declarations do not automatically implement runtime behavior.

For provider operations, inspect the adapter and Java bridge where listed. Moving work to a thread does not automatically make native I/O cancellable. Preserve acknowledgement, session ownership and worker cleanup when changing performance or timeouts.

## Tests and exports

Relevant suites are starting points, not a claim of exhaustive coverage of set\_shared\_variable. Add a regression that executes this exact type/operation with resolved mappings and asserts the returned shape. Exercise empty input, invalid values, provider errors and cancellation as applicable.

```
cd backend
python -m unittest tests.test_mapper_code_general tests.test_raw_python tests.test_engine_export
```

Engine-based export reuses/prunes engine modules. Raw Python export generates task code and specializes raw\_python\_support; review both paths and capability filtering. Re-export a fixture and execute it in a clean test environment. Do not patch only an already-generated archive.

# Tasks, sub-tasks, persistence and copy

Developer guide / Features

## Responsibility

Tasks own activity/transition/group graphs. Calls use saved task references; activity IDs also participate in mapping expressions.

```
Task editor -> normalizeProject -> persistProject -> API -> store.save_project
Call Sub Task -> runtime call_task branch -> child run -> caller output
```

## Source locations

| Responsibility                  | Source location / search anchor                                 |
|---------------------------------|-----------------------------------------------------------------|
| Loaded-project normalization    | frontend/src/main.tsx:1007<br>const normalizeProject =          |
| Browser save API                | frontend/src/main.tsx:2129<br>const persistProject =            |
| Task cloning                    | frontend/src/copyTask.ts:2<br>export function copyTask          |
| Legacy copy reference repair    | frontend/src/repairCopiedTask.ts:1<br>(module)                  |
| Python task schema              | backend/app/models.py:307<br>class ProcessDefinition{           |
| Disk persistence                | backend/app/store.py:46<br>def save_project{                    |
| Nested execution                | backend/app/runtime.py:1116<br>if activity.type == 'call_task': |
| Regression suite: tasks_runtime | backend/tests/test_tasks_runtime.py:1<br>(module)               |

## Safe change procedure

For a new task setting, change both the frontend Task representation and ProcessDefinition validation/defaults. Preserve IDs during a copy or rewrite references consistently. Test nested calls, failed children, interfaces, old imports and missing referenced tasks. A renamed label must not silently break expressions.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Run, continuous events and Stop

Developer guide / Features

## Responsibility

Run coordinates application listeners/jobs while WorkflowRuntime schedules activities and groups.

```
main.tsx run -> POST /projects/{id}/run -> listener registration or runtime.run
Stop -> stop_project -> cancel jobs -> connector cleanup
```

## Source locations

| Responsibility                    | Source location / search anchor                                  |
|-----------------------------------|------------------------------------------------------------------|
| Run action                        | frontend/src/main.tsx:2301<br>const run = async                  |
| Run API                           | backend/app/main.py:617<br>async def run(project_id:             |
| Broker/event polling lifecycle    | backend/app/main.py:292<br>async def _continuous_event_loop(     |
| SAP event lifecycle               | backend/app/main.py:438<br>async def _continuous_sap_event_loop( |
| Stop endpoint                     | backend/app/main.py:666<br>async def stop_project(               |
| Graph execution                   | backend/app/runtime.py:355<br>async def run(self,                |
| Packaged engine command-line host | backend/run_deployment.py:1<br>(module)                          |
| Regression suite: tasks_runtime   | backend/tests/test_tasks_runtime.py:1<br>(module)                |
| Regression suite: publisher_reuse | backend/tests/test_publisher_reuse.py:1<br>(module)              |

## Safe change procedure

Distinguish a one-shot run from a long-lived event starter. Preserve transport context and acknowledgement timing when spawning jobs. For Stop, test active I/O, idle listeners and worker cleanup. Do not equate cancelling an asyncio task with undoing an external send.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Debug controls, test bench and Job Data

Developer guide / Features

## Responsibility

Debug maintains sessions, frames, breakpoints, watches, captured inputs/outputs and isolated activity tests.

```
DebugRibbon / DebugToolsDialog -> debugAction -> DebugManager.action  
-> step / runtime.execute_with_policy -> DebugManager.view -> Job Data
```

## Source locations

| Responsibility                        | Source location / search anchor                           |
|---------------------------------------|-----------------------------------------------------------|
| Start debug request                   | frontend/src/main.tsx:2323<br>debug = async               |
| Debug action transport                | frontend/src/main.tsx:2374<br>debugAction = async         |
| Ribbon controls                       | frontend/src/main.tsx:5890<br>function DebugRibbon(       |
| Test bench/tools                      | frontend/src/main.tsx:5796<br>function DebugToolsDialog(  |
| Input/output/variables viewer         | frontend/src/DebugJobData.tsx:1<br>(module)               |
| Activity-selection tree               | frontend/src/DebugActivityTree.tsx:1<br>(module)          |
| Nested task traversal                 | frontend/src/debugTaskTree.ts:1<br>(module)               |
| Test payload editor                   | frontend/src/DebugPayloadEditor.tsx:1<br>(module)         |
| Debug API                             | backend/app/main.py:1775<br>async def start_debug(        |
| Debug commands                        | backend/app/debugger.py:114<br>async def action(          |
| Frame execution                       | backend/app/debugger.py:274<br>async def step(            |
| Session cleanup                       | backend/app/debugger.py:249<br>async def stop(            |
| Serializable debug state              | backend/app/debugger.py:388<br>def view(                  |
| Resolved input/output capture         | backend/app/runtime.py:550<br>def record_activity_output( |
| Regression suite: debug_testing       | backend/tests/test_debug_testing.py:1<br>(module)         |
| Regression suite: debug_input_capture | backend/tests/test_debug_input_capture.py:1<br>(module)   |

## Safe change procedure

For a new debug action update the client request, DebugAction model and DebugManager.action validation together. Capture resolved activity input separately from previous output. Test pause/resume, nested frames, injected data, run-once side effects and stop during I/O. Test bench execution can modify real systems.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Input mapping, schema mapper and suggestions

Developer guide / Features

## Responsibility

The generic input editor and the dedicated schema mapper are separate UIs; runtime expression resolution and structural mapping are also distinct layers.

```
DataSourcePane -> MappingBinding / InputEditor -> config.inputMappings  
resolve_activity_config -> map_input_values -> evaluate_mapping / resolve  
Mapper activity -> mapper.execute -> output validation
```

## Source locations

| Responsibility                        | Source location / search anchor                                                  |
|---------------------------------------|----------------------------------------------------------------------------------|
| Available upstream Data tree          | frontend/src/ActivityEditor.tsx:1870<br>export function upstreamActivitySources( |
| Data/functions/properties browser     | frontend/src/ActivityEditor.tsx:1918<br>export function DataSourcePane(          |
| Target field control                  | frontend/src/ActivityEditor.tsx:1995<br>function MappingBinding(                 |
| Expanded mapping popup                | frontend/src/ActivityEditor.tsx:2100<br>function ExpandedInputMappingDialog(     |
| Dedicated schema mapper               | frontend/src/MapperStudio.tsx:77<br>export default function MapperStudio         |
| Completion UI/keyboard behavior       | frontend/src/MappingExpressionInput.tsx:1<br>(module)                            |
| Suggestion matching                   | frontend/src/mappingCompletion.ts:1<br>export function mappingPathSuggestions    |
| Input mapping evaluation              | backend/app/runtime.py:1468<br>def map_input_values(                             |
| Structured rule evaluation            | backend/app/runtime.py:1475<br>def evaluate_mapping(                             |
| Reference/expression resolution       | backend/app/runtime.py:1549<br>def resolve(self,                                 |
| Transformation mapping                | backend/app/mapper.py:410<br>def execute(                                        |
| Map & Test endpoint                   | backend/app/main.py:1650<br>def mapper_test(                                     |
| Regression suite: mapper_code_general | backend/tests/test_mapper_code_general.py:1<br>(module)                          |
| Regression suite: sap_idoc_arrays     | backend/tests/test_sap_idoc_arrays.py:1<br>(module)                              |

## Safe change procedure

For a new expression syntax update completion/parser/runtime together. Preserve structured mapping rules and array cardinality; never turn expression strings into unrestricted eval. Compare Map & Test with Run, Debug and direct-export behavior using the same fixture.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Built-in and project functions

Developer guide / Features

## Responsibility

The function catalog controls discoverability; Python evaluation controls results. A catalog entry alone is not executable functionality.

```
mapper-functions.ts template -> expression mapping -> evaluate_function_expression
-> mapper.apply_function / project custom-function expression
```

## Source locations

| Responsibility                              | Source location / search anchor                                             |
|---------------------------------------------|-----------------------------------------------------------------------------|
| All builtin catalog entries                 | frontend/src/mapper-functions.ts:1<br>(module)                              |
| Function picker and custom function editing | frontend/src/ActivityEditor.tsx:1918<br>export function DataSourcePane(<br> |
| Builtin implementation dispatch             | backend/app/mapper.py:195<br>def apply_function(<br>                        |
| Function expression evaluation              | backend/app/runtime.py:1528<br>def evaluate_function_expression(<br>        |
| Persisted custom functions                  | backend/app/models.py:365<br>class Project(<br>                             |
| Export function usage                       | backend/app/raw_python.py:576<br>def raw_python_files(<br>                  |
| Regression suite: mapper_code_general       | backend/tests/test_mapper_code_general.py:1<br>(module)                     |
| Regression suite: raw_python                | backend/tests/test_raw_python.py:1<br>(module)                              |

## Safe change procedure

Add the editor signature, description and insertion template; implement its Python semantics and input validation; update export dependencies if necessary. Test nulls, coercion, arrays, timezone/encoding and invalid arity. Maintain matching semantics in direct archives.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Canvas and mapper drag-and-drop

Developer guide / Features

## Responsibility

Palette creation, moving canvas nodes, connecting transitions and mapping data are different gestures and data-transfer contracts.

```
Palette onDragStart -> canvas onDrop -> addActivity -> task.activities  
Node pointer drag -> coordinates / selection -> saved position  
DataSourcePane drag -> MappingBinding drop -> inputMappings
```

## Source locations

| Responsibility                  | Source location / search anchor                                              |
|---------------------------------|------------------------------------------------------------------------------|
| Palette drag source             | frontend/src/main.tsx:3022<br>onDragStart={e} =>                             |
| Canvas drop target              | frontend/src/main.tsx:3081<br>onDrop={e} =>                                  |
| Activity creation               | frontend/src/main.tsx:1770<br>const addActivity =                            |
| Editable-control shortcut guard | frontend/src/main.tsx:90<br>function editableControlFromEvent(               |
| Transition geometry             | frontend/src/main.tsx:170<br>function activityWireGeometry(                  |
| Picker insertion                | frontend/src/ActivityPicker.tsx:15<br>export default function ActivityPicker |
| Mapping drag sources            | frontend/src/ActivityEditor.tsx:1918<br>export function DataSourcePane(      |
| Mapping drop targets            | frontend/src/ActivityEditor.tsx:1995<br>function MappingBinding(             |
| Group condition drop target     | frontend/src/main.tsx:3650<br>function GroupExpressionTarget(                |
| Selection visuals               | frontend/src/canvas-selection.css:1<br>(module)                              |

## Safe change procedure

Do not change only the drop handler. Check source MIME/data payload, zoom/scroll coordinate conversion, event propagation, nested-group coordinates and undo/persistence. Preserve keyboard input by excluding editable controls from global shortcuts. Verify drag/drop in browser and packaged desktop, plus mapping into the expanded popup.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Groups: editor, graph boundaries and runtime state

Developer guide / Features

## Responsibility

A group includes visual membership plus scheduling, iteration variables, failure handling and resource lifecycle.

```
GroupEditor -> GroupDefinition -> compile_groups
enter_group_boundaries -> _begin_group -> activities
leave_group_boundaries / retry_failed_group -> _finish_group
```

## Source locations

| Responsibility                       | Source location / search anchor                                 |
|--------------------------------------|-----------------------------------------------------------------|
| Create group                         | frontend/src/main.tsx:1945<br>const addGroup =                  |
| Editor defaults                      | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group configuration                  | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Group rendering                      | frontend/src/groups.css:1<br>(module)                           |
| Saved group schema                   | backend/app/models.py:24<br>class GroupDefinition(              |
| Group graph plans                    | backend/app/runtime.py:142<br>def compile_groups(               |
| Enter resources/conditions/iteration | backend/app/runtime.py:214<br>async def _begin_group(           |
| Commit/rollback/release              | backend/app/runtime.py:278<br>async def _finish_group(          |
| Iteration and nested exit            | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Retry scope                          | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Native transaction session           | backend/app/jdbc.py:270<br>class JavaJdbcTransaction            |
| Direct-export group planning         | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime     | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python         | backend/tests/test_raw_python.py:1<br>(module)                  |

## Safe change procedure

Keep editor defaults, parent\_group\_id/member\_activity\_ids validation and runtime plans aligned. Test nested entry/exit, skip branches, retries, cancellation and restoration of loop variables. Transaction groups must close the exact owned connection on every exit. Critical sections are runtime-local, not distributed locks.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# If: source map

Developer guide / Groups

## Behavior to preserve

Evaluates condition before entry; false skips the group.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                              | Source location / search anchor                                 |
|---------------------------------------------|-----------------------------------------------------------------|
| If defaults; match group type if            | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                  | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch              | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                        | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                            | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning               | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime            | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# While True: source map

Developer guide / Groups

## Behavior to preserve

Evaluates condition before each iteration, including the first.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                              | Source location / search anchor                                     |
|---------------------------------------------|---------------------------------------------------------------------|
| While True defaults; match group type while | frontend/src/main.tsx:3664<br>function groupConfigDefaults(<br>     |
| Group-specific condition/iteration controls | frontend/src/main.tsx:3670<br>function GroupEditor(<br>             |
| Type-specific entry branch                  | backend/app/runtime.py:214<br>async def _begin_group(<br>           |
| Type-specific loop/exit branch              | backend/app/runtime.py:304<br>async def leave_group_boundaries(<br> |
| Error/retry handling                        | backend/app/runtime.py:334<br>async def retry_failed_group(<br>     |
| Resource release                            | backend/app/runtime.py:278<br>async def _finish_group(<br>          |
| Export nesting/graph planning               | backend/app/raw_python.py:370<br>def _simple_group_plans(<br>       |
| Regression suite: groups_runtime            | backend/tests/test_groups_runtime.py:1<br>(module)                  |
| Regression suite: raw_python                | backend/tests/test_raw_python.py:1<br>(module)                      |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# For Each: source map

Developer guide / Groups

## Behavior to preserve

Resolves the collection at entry and runs once per item. Range mode uses inclusive start/end with a nonzero increment.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                               | Source location / search anchor                                 |
|----------------------------------------------|-----------------------------------------------------------------|
| For Each defaults; match group type for-each | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls  | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                   | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch               | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                         | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                             | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning                | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime             | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                 | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Iterate: source map

Developer guide / Groups

## Behavior to preserve

Resolves source once at entry and iterates its elements.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                              | Source location / search anchor                                     |
|---------------------------------------------|---------------------------------------------------------------------|
| Iterate defaults; match group type iterate  | frontend/src/main.tsx:3664<br>function groupConfigDefaults(<br>     |
| Group-specific condition/iteration controls | frontend/src/main.tsx:3670<br>function GroupEditor(<br>             |
| Type-specific entry branch                  | backend/app/runtime.py:214<br>async def _begin_group(<br>           |
| Type-specific loop/exit branch              | backend/app/runtime.py:304<br>async def leave_group_boundaries(<br> |
| Error/retry handling                        | backend/app/runtime.py:334<br>async def retry_failed_group(<br>     |
| Resource release                            | backend/app/runtime.py:278<br>async def _finish_group(<br>          |
| Export nesting/graph planning               | backend/app/raw_python.py:370<br>def _simple_group_plans(<br>       |
| Regression suite: groups_runtime            | backend/tests/test_groups_runtime.py:1<br>(module)                  |
| Regression suite: raw_python                | backend/tests/test_raw_python.py:1<br>(module)                      |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Repeat Until True: source map

Developer guide / Groups

## Behavior to preserve

Runs once, checks condition at exit and repeats until true.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                                      | Source location / search anchor                                 |
|-----------------------------------------------------|-----------------------------------------------------------------|
| Repeat Until True defaults; match group type repeat | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls         | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                          | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch                      | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                                | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                                    | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning                       | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime                    | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Repeat on Error Until True: source map

Developer guide / Groups

## Behavior to preserve

Retries the whole group after an unhandled fault while stopCondition is false. A true stop condition or exhausted retry count propagates the fault.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent\_group\_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                                                        | Source location / search anchor                                 |
|-----------------------------------------------------------------------|-----------------------------------------------------------------|
| Repeat on Error Until True defaults; match group type repeat-on-error | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls                           | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                                            | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch                                        | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                                                  | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                                                      | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning                                         | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime                                      | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                                          | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Scope: source map

Developer guide / Groups

## Behavior to preserve

Runs the enclosed graph once. Nested groups retain their own semantics.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                              | Source location / search anchor                                 |
|---------------------------------------------|-----------------------------------------------------------------|
| Scope defaults; match group type scope      | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                  | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch              | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                        | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                            | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning               | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime            | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# None: source map

Developer guide / Groups

## Behavior to preserve

Runs the enclosed graph once; this is an execution boundary, not a separate transaction.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                              | Source location / search anchor                                     |
|---------------------------------------------|---------------------------------------------------------------------|
| None defaults; match group type none        | frontend/src/main.tsx:3664<br>function groupConfigDefaults(<br>     |
| Group-specific condition/iteration controls | frontend/src/main.tsx:3670<br>function GroupEditor(<br>             |
| Type-specific entry branch                  | backend/app/runtime.py:214<br>async def _begin_group(<br>           |
| Type-specific loop/exit branch              | backend/app/runtime.py:304<br>async def leave_group_boundaries(<br> |
| Error/retry handling                        | backend/app/runtime.py:334<br>async def retry_failed_group(<br>     |
| Resource release                            | backend/app/runtime.py:278<br>async def _finish_group(<br>          |
| Export nesting/graph planning               | backend/app/raw_python.py:370<br>def _simple_group_plans(<br>       |
| Regression suite: groups_runtime            | backend/tests/test_groups_runtime.py:1<br>(module)                  |
| Regression suite: raw_python                | backend/tests/test_raw_python.py:1<br>(module)                      |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Critical Section: source map

Developer guide / Groups

## Behavior to preserve

Holds a runtime-local named lock until completion, failure or cancellation. This is not a distributed lock across machines or separate runtime processes.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                                               | Source location / search anchor                                 |
|--------------------------------------------------------------|-----------------------------------------------------------------|
| Critical Section defaults; match group type critical-section | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls                  | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                                   | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch                               | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                                         | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                                             | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning                                | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime                             | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                                 | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# JDBC Transaction: source map

Developer guide / Groups

## Behavior to preserve

Uses one database connection for matching JDBC operations, commits on successful exit and rolls back on an escaping fault or cancellation. External non-JDBC side effects are not rolled back.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                                               | Source location / search anchor                                 |
|--------------------------------------------------------------|-----------------------------------------------------------------|
| JDBC Transaction defaults; match group type transaction-jdbc | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls                  | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                                   | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch                               | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                                         | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                                             | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning                                | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime                             | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                                 | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Pick First: source map

Developer guide / Groups

## Behavior to preserve

Chooses the first eligible transition in graph order. This is first-match routing, not concurrent event racing or fastest-response selection.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Use the group type branch in these shared methods. Do not create a UI-only group option; update validation, runtime entry/exit and direct-export scheduling together.

## Source locations

| Responsibility                                      | Source location / search anchor                                 |
|-----------------------------------------------------|-----------------------------------------------------------------|
| Pick First defaults; match group type<br>pick-first | frontend/src/main.tsx:3664<br>function groupConfigDefaults(     |
| Group-specific condition/iteration controls         | frontend/src/main.tsx:3670<br>function GroupEditor(             |
| Type-specific entry branch                          | backend/app/runtime.py:214<br>async def _begin_group(           |
| Type-specific loop/exit branch                      | backend/app/runtime.py:304<br>async def leave_group_boundaries( |
| Error/retry handling                                | backend/app/runtime.py:334<br>async def retry_failed_group(     |
| Resource release                                    | backend/app/runtime.py:278<br>async def _finish_group(          |
| Export nesting/graph planning                       | backend/app/raw_python.py:370<br>def _simple_group_plans(       |
| Regression suite: groups_runtime                    | backend/tests/test_groups_runtime.py:1<br>(module)              |
| Regression suite: raw_python                        | backend/tests/test_raw_python.py:1<br>(module)                  |

## Regression focus

Test zero/one/multiple iterations or selected branches as applicable; nested groups; an activity fault; cancellation; and a saved/archive round trip. Verify variables, locks and sessions are restored or released at scope exit.

# Engine archives versus direct Python generation

Developer guide / Features

## Responsibility

The two archive forms have different code-generation and packaging responsibilities.

```
PackageDialog -> package_project -> build_deployment_archive
engine_python_files -> pruned engine modules
raw_python_files -> task Python + specialized support modules
```

## Source locations

| Responsibility                  | Source location / search anchor                                         |
|---------------------------------|-------------------------------------------------------------------------|
| Packaging UI                    | frontend/src/main.tsx:3746<br>function PackageDialog(                   |
| Archive orchestration           | backend/app/main.py:1252<br>def build_deployment_archive(               |
| Capability registry             | backend/app/raw_python.py:60<br>def _activity_capabilities(             |
| Direct application generation   | backend/app/raw_python.py:576<br>def raw_python_files(                  |
| Engine archive generation       | backend/app/raw_python.py:1151<br>def engine_python_files(              |
| Engine module pruning           | backend/app/raw_python.py:1097<br>def _engine_module_files(             |
| Direct support dispatch         | backend/app/raw_python_support/core.py:182<br>async def execute(        |
| Direct application lifecycle    | backend/app/raw_python_support/main.py:24<br>async def run_application( |
| Regression suite: raw_python    | backend/tests/test_raw_python.py:1<br>(module)                          |
| Regression suite: engine_export | backend/tests/test_engine_export.py:1<br>(module)                       |

## Safe change procedure

Changing runtime.py alone is insufficient when direct archives use raw\_python\_support. Update capability roots and generated import closure without adding unrelated helpers to every project. Validate a minimal one-activity archive, a mixed project, nested groups and a clean-environment execution. Keep vendor files and secrets out of generic archives.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Logs, faults and observable execution data

Developer guide / Features

## Responsibility

Log activity output, activity-policy payload logs and persistent project log files are separate responsibilities.

```
execute_with_policy / Log activity -> runtime.log -> project_logging
API runtime-state / logs -> Studio panels
```

## Source locations

| Responsibility                        | Source location / search anchor                          |
|---------------------------------------|----------------------------------------------------------|
| Runtime event emission                | backend/app/runtime.py:570<br>def log(self,              |
| Log activity body                     | backend/app/runtime.py:816<br>if activity.type == 'log': |
| Fault envelope                        | backend/app/runtime.py:566<br>def fault_payload(         |
| Persistent log files/rotation         | backend/app/project_logging.py:1<br>(module)             |
| Logs API                              | backend/app/main.py:589<br>def project_logs(             |
| Regression suite: project_logging     | backend/tests/test_project_logging.py:1<br>(module)      |
| Regression suite: debug_input_capture | backend/tests/test_debug_input_capture.py:1<br>(module)  |

## Safe change procedure

Keep secrets and large payloads out of logs by default. Preserve timestamps, activity identity, error codes, environment path selection and rolling-file behavior. Test write permissions and failure logging as well as successful execution.

Add tests at the affected boundary, not only the UI. Regenerate this source guide after edits: anchors are checked during docs:build and participating source changes invalidate docs:check.

# Every built-in function: implementation index

Developer guide / Functions

## Where to edit

Every catalog function is listed below. Most built-ins share `apply_function` rather than having one Python function per name. Search the quoted function name or its aliases inside `apply_function`; changing the catalog does not add execution support. Runtime expression parsing and custom functions also pass through `evaluate_function_expression`.

| Responsibility          | Source location / search anchor                                               |
|-------------------------|-------------------------------------------------------------------------------|
| Implementation/aliases  | backend/app/mapper.py:195<br><code>def apply_function(</code>                 |
| Expression call parsing | backend/app/runtime.py:1528<br><code>def evaluate_function_expression(</code> |
| Signatures/templates    | frontend/src/mapper-functions.ts:1<br>(module)                                |

## Catalog index

| Function / signature                                                       | Frontend definition / backend entry                                                                                                               |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>concat</code><br><code>concat(value, ...values)</code>               | frontend/src/mapper-functions.ts:13<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>concat</code>          |
| <code>trim</code><br><code>trim(value)</code>                              | frontend/src/mapper-functions.ts:14<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>trim</code>            |
| <code>normalizeSpace</code><br><code>normalizeSpace(value)</code>          | frontend/src/mapper-functions.ts:15<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>normalizeSpace</code>  |
| <code>upperCase</code><br><code>upperCase(value)</code>                    | frontend/src/mapper-functions.ts:16<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>upperCase</code>       |
| <code>lowerCase</code><br><code>lowerCase(value)</code>                    | frontend/src/mapper-functions.ts:17<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>lowerCase</code>       |
| <code>capitalize</code><br><code>capitalize(value)</code>                  | frontend/src/mapper-functions.ts:18<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>capitalize</code>      |
| <code>stringLength</code><br><code>stringLength(value)</code>              | frontend/src/mapper-functions.ts:19<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>stringLength</code>    |
| <code>substring</code><br><code>substring(value, start, length?)</code>    | frontend/src/mapper-functions.ts:20<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>substring</code>       |
| <code>substringBefore</code><br><code>substringBefore(value, token)</code> | frontend/src/mapper-functions.ts:21<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>substringBefore</code> |
| <code>substringAfter</code><br><code>substringAfter(value, token)</code>   | frontend/src/mapper-functions.ts:22<br>backend/app/mapper.py :: <code>apply_function</code><br>Search literal/alias: <code>substringAfter</code>  |

| Function / signature                             | Frontend definition / backend entry                                                                                     |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| replace<br>replace(value, pattern, replacement)  | frontend/src/mapper-functions.ts:23<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: replace         |
| matches<br>matches(value, pattern)               | frontend/src/mapper-functions.ts:24<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: matches         |
| startsWith<br>startsWith(value, prefix)          | frontend/src/mapper-functions.ts:25<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: startsWith      |
| endsWith<br>endsWith(value, suffix)              | frontend/src/mapper-functions.ts:26<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: endsWith        |
| contains<br>contains(value, token)               | frontend/src/mapper-functions.ts:27<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: contains        |
| compare<br>compare(value, other)                 | frontend/src/mapper-functions.ts:28<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: compare         |
| translate<br>translate(value, from, to)          | frontend/src/mapper-functions.ts:29<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: translate       |
| padLeft<br>padLeft(value, width, fill?)          | frontend/src/mapper-functions.ts:30<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: padLeft         |
| padRight<br>padRight(value, width, fill?)        | frontend/src/mapper-functions.ts:31<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: padRight        |
| tokenize<br>tokenize(value, pattern)             | frontend/src/mapper-functions.ts:32<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: tokenize        |
| currentDate<br>currentDate()                     | frontend/src/mapper-functions.ts:34<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: currentDate     |
| currentTime<br>currentTime()                     | frontend/src/mapper-functions.ts:35<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: currentTime     |
| currentDateTime<br>currentDateTime()             | frontend/src/mapper-functions.ts:36<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: currentDateTime |
| parseDate<br>parseDate(value)                    | frontend/src/mapper-functions.ts:37<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: parseDate       |
| formatDate<br>formatDate(value, pattern)         | frontend/src/mapper-functions.ts:38<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: formatDate      |
| formatDateTime<br>formatDateTime(value, pattern) | frontend/src/mapper-functions.ts:39<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: formatDateTime  |

| Function / signature                                | Frontend definition / backend entry                                                                                          |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| addDays<br>addDays(value, days)                     | frontend/src/mapper-functions.ts:40<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: addDays              |
| addMonths<br>addMonths(value, months)               | frontend/src/mapper-functions.ts:41<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: addMonths            |
| addHours<br>addHours(value, hours)                  | frontend/src/mapper-functions.ts:42<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: addHours             |
| addMinutes<br>addMinutes(value, minutes)            | frontend/src/mapper-functions.ts:43<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: addMinutes           |
| addSeconds<br>addSeconds(value, seconds)            | frontend/src/mapper-functions.ts:44<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: addSeconds           |
| dateDifference<br>dateDifference(from, to)          | frontend/src/mapper-functions.ts:45<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: dateDifference       |
| year<br>year(value)                                 | frontend/src/mapper-functions.ts:46<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: year                 |
| month<br>month(value)                               | frontend/src/mapper-functions.ts:46<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: month                |
| day<br>day(value)                                   | frontend/src/mapper-functions.ts:46<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: day                  |
| hour<br>hour(value)                                 | frontend/src/mapper-functions.ts:46<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: hour                 |
| minute<br>minute(value)                             | frontend/src/mapper-functions.ts:46<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: minute               |
| second<br>second(value)                             | frontend/src/mapper-functions.ts:46<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: second               |
| timezoneFromDateTime<br>timezoneFromDateTime(value) | frontend/src/mapper-functions.ts:47<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: timezoneFromDateTime |
| number<br>number(value)                             | frontend/src/mapper-functions.ts:49<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: number               |
| integer<br>integer(value)                           | frontend/src/mapper-functions.ts:50<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: integer              |
| round<br>round(value, precision?)                   | frontend/src/mapper-functions.ts:51<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: round                |

| Function / signature                                  | Frontend definition / backend entry                                                                                     |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| roundHalfToEven<br>roundHalfToEven(value, precision?) | frontend/src/mapper-functions.ts:52<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: roundHalfToEven |
| floor<br>floor(value)                                 | frontend/src/mapper-functions.ts:53<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: floor           |
| ceiling<br>ceiling(value)                             | frontend/src/mapper-functions.ts:54<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: ceiling         |
| abs<br>abs(value)                                     | frontend/src/mapper-functions.ts:55<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: abs             |
| sqrt<br>sqrt(value)                                   | frontend/src/mapper-functions.ts:56<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: sqrt            |
| power<br>power(value, exponent)                       | frontend/src/mapper-functions.ts:57<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: power           |
| modulo<br>modulo(value, divisor)                      | frontend/src/mapper-functions.ts:58<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: modulo          |
| clamp<br>clamp(value, minimum, maximum)               | frontend/src/mapper-functions.ts:59<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: clamp           |
| min<br>min(values)                                    | frontend/src/mapper-functions.ts:60<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: min             |
| max<br>max(values)                                    | frontend/src/mapper-functions.ts:60<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: max             |
| sum<br>sum(values)                                    | frontend/src/mapper-functions.ts:60<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: sum             |
| average<br>average(values)                            | frontend/src/mapper-functions.ts:60<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: average         |
| count<br>count(values)                                | frontend/src/mapper-functions.ts:60<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: count           |
| distinctValues<br>distinctValues(values)              | frontend/src/mapper-functions.ts:62<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: distinctValues  |
| sort<br>sort(values, descending?)                     | frontend/src/mapper-functions.ts:63<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: sort            |
| reverse<br>reverse(values)                            | frontend/src/mapper-functions.ts:64<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: reverse         |

| Function / signature                                | Frontend definition / backend entry                                                                                 |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| first<br>first(values)                              | frontend/src/mapper-functions.ts:65<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: first       |
| last<br>last(values)                                | frontend/src/mapper-functions.ts:66<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: last        |
| indexOf<br>indexOf(values, item)                    | frontend/src/mapper-functions.ts:67<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: indexOf     |
| subsequence<br>subsequence(values, start, end?)     | frontend/src/mapper-functions.ts:68<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: subsequence |
| flatten<br>flatten(values)                          | frontend/src/mapper-functions.ts:69<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: flatten     |
| join<br>join(values, separator)                     | frontend/src/mapper-functions.ts:70<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: join        |
| split<br>split(value, pattern)                      | frontend/src/mapper-functions.ts:71<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: split       |
| deepEqual<br>deepEqual(value, other)                | frontend/src/mapper-functions.ts:72<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: deepEqual   |
| string<br>string(value)                             | frontend/src/mapper-functions.ts:74<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: string      |
| boolean<br>boolean(value)                           | frontend/src/mapper-functions.ts:75<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: boolean     |
| jsonParse<br>jsonParse(value)                       | frontend/src/mapper-functions.ts:76<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: jsonParse   |
| jsonRender<br>jsonRender(value)                     | frontend/src/mapper-functions.ts:77<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: jsonRender  |
| render-xml<br>render-xml(value, rootName?, pretty?) | frontend/src/mapper-functions.ts:78<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: render-xml  |
| coalesce<br>coalesce(value, ...fallbacks)           | frontend/src/mapper-functions.ts:80<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: coalesce    |
| default<br>default(value, fallback)                 | frontend/src/mapper-functions.ts:81<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: default     |
| exists<br>exists(value)                             | frontend/src/mapper-functions.ts:82<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: exists      |

| Function / signature                                       | Frontend definition / backend entry                                                                                  |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| empty<br>empty(value)                                      | frontend/src/mapper-functions.ts:83<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: empty        |
| not<br>not(value)                                          | frontend/src/mapper-functions.ts:84<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: not          |
| ifThenElse<br>ifThenElse(condition, trueValue, falseValue) | frontend/src/mapper-functions.ts:85<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: ifThenElse   |
| uuid<br>uuid()                                             | frontend/src/mapper-functions.ts:86<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: uuid         |
| base64Encode<br>base64Encode(value)                        | frontend/src/mapper-functions.ts:88<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: base64Encode |
| base64Decode<br>base64Decode(value)                        | frontend/src/mapper-functions.ts:89<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: base64Decode |
| hexEncode<br>hexEncode(value)                              | frontend/src/mapper-functions.ts:90<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: hexEncode    |
| hexDecode<br>hexDecode(value)                              | frontend/src/mapper-functions.ts:91<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: hexDecode    |
| urlEncode<br>urlEncode(value)                              | frontend/src/mapper-functions.ts:92<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: urlEncode    |
| urlDecode<br>urlDecode(value)                              | frontend/src/mapper-functions.ts:93<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: urlDecode    |
| hash<br>hash(value, algorithm?)                            | frontend/src/mapper-functions.ts:94<br>backend/app/mapper.py :: apply_function<br>Search literal/alias: hash         |